

Research Recorder and Test Automation — Operator Manual

What this technology does

The research recorder is the authoritative evidence-capture system for the dissertation experiments. It captures USB source attempts, Gateway-01 evidence, cloud logs, resource metrics, database exports, evidence status, Fabric outbox state, run timing, hashes and derived summaries.

Grafana and the live cockpit are observation aids. The sealed recorder bundle is the authority for a formal trial.

Current entry points

```
test/automation/research-recorder/research_recorder.py
test/automation/research-manual/run_test.py
test/automation/research-manual/ensure_ready.py
test/automation/research-recorder/research_cockpit.py
test/automation/research-recorder/summarize-run.py
test/automation/research-recorder/summarize-study.py
```

The current repository server forced-command wrapper identifies itself as research-recorder-server-v7. Older v5 prose is historical; always verify live wrapper compatibility through current preflight/version checks.

Step 1 — Read the current authority

Before a formal test read:

```
test/ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md
test/automation/research-manual/CURRENT-READINESS.md
test/automation/research-manual/CONTINUATION.md
chapters/lorawan_research_test_manual_final.md
```

Chapter requirements define what must be measured. Verified live infrastructure defines how it is executed.

Step 2 — Install the pinned Python dependency

```
python -m pip install --user -r .\lorawan-network-server-gateway\test\automation\research-
recorder\requirements.txt
```

The pinned dependency is pyserial 3.5.

Do not weaken workstation PowerShell policy to run older prototypes.

Step 3 — Preserve the restricted SSH boundary

Recorder key:

```
%USERPROFILE%\ssh\id_ed25519_research_recorder
```

The key is forced-command only. It must not provide arbitrary shell access. Never replace it with an embedded password or administrator private key.

Step 4 — Check recorder state

```
$R = '.\lorawan-network-server-gateway\test\automation\research-recorder\research_recorder.py'; python $R status
```

PASS means: no unexpected formal run is active.

If an interrupted run exists, preserve it and use recover; do not delete its marker.

Step 5 — Run formal preflight

```
python $R preflight --require-emu --require-sec
```

This checks serial devices, cloud collectors, database topology, evidence services, clock alignment and required tooling.

A green platform preflight does not make every experiment READY.

Step 6 — Respect the clock gate

Do not start timing-sensitive counted work while recorder clock validation fails.

If it fails:

1. determine whether workstation/PDC, Gateway-01 or cloud time is wrong;
2. repair the authoritative time source;
3. rerun preflight;
4. begin no counted run until PASS.

Never rewrite timestamps after capture.

Step 7 — Check formal test readiness

```
python .\lorawan-network-server-gateway\test\automation\research-manual\run_test.py --help
```

Status meanings:

READY	complete runnable action
CAPTURE_READY	capture exists; deliberate fixture incomplete
HARNESS_REQUIRED	reviewed orchestrator/fixture still missing
METHODOLOGY_REQUIRED	Chapter 3 itself is incomplete

Only READY tests may appear as executable blocks in the final manual.

Step 8 — Start a formal recorder run

Example one-line pattern:

```
python $R run --group authentication --run-id auth-lorawan-A1-01 --condition 'registered device with correct OTAA credentials' --expected 'allow; JoinAccept and accepted uplink' --scope lorawan --capture-sec
```

Keep the foreground recorder terminal open. Use the exact experiment group/run ID required by the final manual.

Step 9 — Mark deliberate actions

```
python $R mark --message 'A1-01 reset issued'
```

Use markers for resets, cable changes, fault injection or physical actions whose timing matters.

Step 10 — Yield one serial port safely

EMU:

```
python $R serial-pause --device emu
python $R serial-resume --device emu
```

SEC:

```
python $R serial-pause --device sec
python $R serial-resume --device sec
```

Perform only the approved fixture/firmware action between pause and resume.

Step 11 — Inspect evidence state while running

```
python $R evidence-status
```

This is a fixed read-only boundary. Do not manually rewrite verifier or Fabric states to improve a trial.

Step 12 — Record trial classification after evidence review

```
python $R trial --trial-id A1-01 --condition 'correct OTAA credentials' --expected allow --actual allow --status PASS --notes 'JoinRequest received; JoinAccept; application uplink stored'
```

Allowed classifications: PASS, FAIL, INVALID and BLOCKED.

The recorder itself finalizes as RECORDED_UNCLASSIFIED; it does not invent the conclusion.

Step 13 — Stop normally

Use Ctrl+C in the foreground run.

Normal finalization must:

```
stop collectors
export exact-window DB/log evidence
calculate derived metrics
write end metadata
hash completed evidence
seal raw files read-only
remove active marker
```

Do not kill Python when clean finalization is possible.

Step 14 — Recover an interrupted run

```
python $R status
python $R recover
```

Recovery preserves and finalizes partial evidence. Do not delete `_recorder-active.json` to get back to CLEAN.

Step 15 — Understand the evidence tree

```
chapter4-results/<group>/<run-id>/
  raw/
  metadata/
  derived/
```

Typical authoritative files include source serial, gateway/resource data, ULC resource data, `uplinks.csv`, `measurements.csv`, `fabric-outbox.csv`, exact-window service logs, run metadata, `SHA256SUMS.csv` and derived trial summaries.

Never edit raw files.

Step 16 — Verify the evidence seal

A valid finalized run must have:

```
no missing manifest files
no hash mismatch
no unexplained active marker
no unexplained stop/supervisor failure
all required collectors present
```

If a required stream failed, classify the trial from evidence; do not repair raw files afterward.

Step 17 — Keep PDR and database delivery separate

Formal Chapter 3 PDR:

```
unique legitimate ChirpStack-accepted uplinks / captured scheduled SENSOR_TX attempts x 100
```

Database delivery:

```
exact DB payload matches for comparable source attempts / eligible comparable source attempts x 100
```

Never use simple DB-row-count/source-count as PDR.

Step 18 — Preserve exact payload correlation

The recorder reconstructs the 46-byte payload-v2 from each SENSOR_TX line and matches it to telemetry.uplinks.raw_data.

Do not replace exact-byte correlation with timestamp-only, FCnt-only or sequence-only matching.

Step 19 — Use the cockpit correctly

```
http://127.0.0.1:8765/
```

The cockpit shows packet flow, resources, service roles, database counts, evidence readiness and Fabric state.

It is not the formal result authority. Sealed recorder evidence is.

Step 20 — Use skip-serial only for uncounted smoke

```
python $R run --skip-serial --duration-seconds 60 --group smoke --run-id recorder-smoke
```

Never use --skip-serial when the formal denominator depends on SENSOR_TX attempts.

Current restricted remote verbs

```
version
utc-now
clock-probe
evidence-ready
snapshot
db-role
evidence-status
monitor-stream
resource-host
resource-docker
db-export
docker-log
unit-log
kernel-log
```

Arbitrary commands must remain rejected.

Troubleshooting order

Symptom	First check
EMU COM busy	Serial Monitor/orphan serial process
SEC missing	current COM / device mode

Symptom	First check
clock gate fails	PDC/workstation/Gateway/cloud time
remote command rejected	wrapper version/deployment
one collector dies	supervisor and that collector log
no DB rows	source -> RF -> ChirpStack -> app
DB rows, no Fabric terminal state	evidence eligibility/finalization/adaptor
impossible >100% PDR	metric/window/deduplication bug
hash mismatch	preserve evidence and classify honestly
cockpit stale	monitor stream; never edit evidence

Security boundary

- recorder SSH is forced-command only;
- no arbitrary SQL from workstation;
- no embedded admin password;
- AppKeys/private keys must not enter evidence;
- raw evidence is sealed read-only;
- failures are preserved rather than cleaned away.

Completion checklist

- current source-of-truth/readiness read;
- recorder clean before start;
- preflight PASS;
- clock gate PASS;
- required serial sources captured;
- supervisor remains healthy;
- actions marked;
- classification evidence-based;
- run finalizes/recover succeeds;
- manifest hashes verify;
- raw evidence read-only;
- correct metric definitions used;
- non-ready tests remain unpublished.

Detailed references

- [Recorder README](#)
- [Research manual tooling](#)
- [Continuation state](#)
- [Metrics source of truth](#)

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Automated Research Recorder

This is the preferred evidence-capture path for counted dissertation tests on the **commissioned** Gateway-01 + LTE + three-node cloud system.

Evidence authority

The testing evidence is authoritative for observed results. The current methodology authority for what must be tested and measured is [.././ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md](#), derived from the new chapters/Zacarias_Chapter3.pdf. Use it for research questions, experimental conditions, counts, durations, measurements, analysis rules, and stated acceptance/security criteria. Do not treat architecture, topology, technology placement, host roles, status statements, or expected outcomes in that PDF—or older dissertation drafts—as current infrastructure truth.

Use this order when sources disagree:

1. sealed evidence captured for the specific formal trial under chapter4-results/ is the authority for that trial and its research classification;
2. fresh read-only live observations are the authority for current operational state at the time they were observed;
3. current deployment/configuration material is the reproducibility reference;
4. chapter PDFs, old DOCX files and historical prose are requirements/provenance context only.

A healthy live service does **not** prove that a research trial passed. The recorder deliberately captures evidence first and leaves classification to the evidence from that trial. The implementation must follow the live commissioned system rather than recreating obsolete architecture described in older drafts.

What is automated

One recorder run automatically retains:

- EMU-01 USB serial with host UTC timestamps and automatic COM-port re-enumeration recovery;
- optional SEC USB serial capture;
- Gateway-01 live logread stream;
- Gateway-01 CPU, memory, load and network counters;
- Gateway-01 LTE/QMI, route and production MQTT-socket snapshots;
- host CPU, memory, load and network counters on ulc-01, ulc-02, ulc-03;
- per-container CPU/memory series on all three cloud nodes;
- exact-window ChirpStack, Node-RED, Mosquitto and kernel logs;

- authoritative `telemetry.uplinks`, `telemetry.measurements`, and `telemetry.fabric_outbox` CSV exports from the current Patroni leader;
- RSSI, SNR, gateway frequency, gateway uplink ID, FCnt and `test_sequence` statistics;
- byte-exact EMU-01 source-to-database **delivery correlation** whose source timestamps can be clock-aligned into the authoritative DB export window; this is not the formal Chapter 3 PDR;
- formal Chapter 3 PDR is reported only when the numerator can be derived as unique legitimate uplinks accepted by ChirpStack over the same eligible source-attempt window;
- application-event-to-database timing (`received_at - time`) plus independently calibrated **sensor-transmission-to-database-storage latency** from matched `SENSOR_TX` source timestamps when recorder-v9 start/end clock calibration passes;
- a read-only aggregate verifier/checkpoint snapshot at run start and run end (`metadata/evidence-status-start.txt` and `metadata/evidence-status-end.txt`);
- SHA-256 for every completed evidence file.

Raw files are made read-only after finalization. Calculated files live under `derived/`; never edit raw evidence to improve a result.

Live testing monitor

`research_cockpit.py` provides the read-only testing monitor at `http://127.0.0.1:8765/`. It is an observation aid for running experiments, not a second result engine.

The cockpit combines the existing live sensor/resource view with `testing_monitor.py`. Each cloud node uses one locked-down SSH `monitor-stream` session so monitoring does not create a burst of competing SSH probes during a trial. Resource records remain approximately one-second observations; slower service/database/evidence observations are multiplexed through the same session. The server wrapper discovers the live Docker/service set instead of relying on chapter-era service lists.

The monitor shows:

- current CPU utilization as percent of 100%, RAM used/total, and network receive/send rates in Mbps with explicit capacity context where a defensible capacity is known;
- received sensor packets by sender, DevEUI, gateway, AS923 radio frequency, RSSI in dBm, SNR in dB, FCnt/test sequence and decoded sensor values;
- packet-level RF-to-application correlation from Gateway-01 reception through gateway MQTT evidence, ChirpStack/application acceptance, TimescaleDB, evidence verification, Fabric outbox and Fabric transaction fields when present;
- duplicate/replay indicators that distinguish an exact PHYPayload received again under a new gateway uplink ID from ordinary MQTT redelivery, plus FCnt/test-sequence repeats, gaps and regressions;
- discovered containers and selected system services with node, role/state and live detail;
- current Patroni leader/replica observations;
- evidence collector/verifier and Fabric adapter readiness, including the deliberately fenced Fabric standby when observed;
- leader-only database counters for LoRaWAN uplinks, decoded measurements, gateway MQTT evidence, evidence segments, checkpoints, verifications and Fabric outbox rows;
- recent warning/error observations and the formal test inventory from `chapter4-results/`.

Operational observation history is stored locally in `%LOCALAPPDATA%\LoRaWAN\testing-monitor.sqlite3` using SQLite/WAL. Packet observations are stored separately in `%LOCALAPPDATA%\LoRaWAN\packet-monitor.sqlite3`. Service snapshots are persisted at a bounded cadence (currently 60 seconds), packet correlation is refreshed every five seconds, the UI exposes recent history, and observations are retained for seven days. These local

histories are useful for diagnosing service and packet behavior; they are **not** substituted for the sealed raw/metadata/derived files of a formal trial.

The packet monitor deliberately uses three different duplicate concepts. An exact `phy_payload_sha256` observed with a **new gateway uplink ID** is a strong RF replay/duplicate candidate. Re-observation of the same gateway uplink ID is transport/capture redelivery and is not counted as another RF packet. Duplicate application identity, `FCnt` or `test_sequence` is a downstream/session anomaly and is reported separately. This distinction prevents QoS 1 MQTT behavior from being misreported as a LoRaWAN replay.

Clock and timing gate

Counted runs fail closed unless the source-clock relationship to authoritative cloud UTC is **measured and stable**. `serial_capture.py` timestamps each EMU-01 `SENSOR_TX` line with the workstation clock, while the authoritative run window and database evidence use cloud/server UTC. Recorder v9 therefore does not assume that the workstation wall clock is correct: it independently calibrates the workstation against `ulc-01` immediately before the formal window, repeats the calibration after collectors stop, and invalidates finalization if the start/end mapping drifts by more than **0.100 s**. The calibrated mapping is retained in `metadata/run-meta.json` and is used to derive the Chapter 3 sensor-transmission-to-database-storage latency.

The recorder preflight still takes five midpoint-corrected `utc-now` samples from each of `ulc-01`, `ulc-02`, and `ulc-03`. The three cloud clocks must agree within **0.250 s**, Gateway-01 must remain within **±1.500 s** of the cloud median, and the workstation/cloud offset must be measurable within the bounded calibration range (**±300 s**). Absolute workstation offset is recorded rather than silently treated as zero. A large offset is acceptable only when start/end calibration remains stable; cloud disagreement, gateway disagreement, missing calibration, excessive drift, or negative calibrated sensor-to-database latency invalidates timing evidence.

Historical staging diagnosis found that the domain workstation could inherit a large absolute offset from its PDC even while the cloud and gateway clocks remained mutually coherent. Recorder v9 records that offset explicitly and proves its stability at both run boundaries, so a large but stable workstation wall-clock offset no longer blocks a run or contaminates source timing. The Active Directory time source should still be repaired through the normal PDC hierarchy for operational correctness; do not point a domain member directly at public NTP merely to satisfy research tooling.

Gateway-01 remains independently gated against the cloud median because gateway radio/evidence timestamps are part of the cross-system observation boundary. Its limit remains **±1.500 s**. If gateway/cloud agreement or cloud-node spread exceeds its limit, the recorder still fails closed. The restricted research-recorder key remains read-only and must not be weakened to repair time service configuration.

The cockpit's **Overall staging readiness** panel combines this clock gate with live cloud streams, Gateway management visibility, Patroni role topology, evidence readiness, leader database counters, telemetry-read availability, and the absence of an already-active formal recorder. This panel is an operational preflight aid only; the recorder CLI remains the final start gate and additionally checks required local USB/serial devices.

The server side remains read-only. `DBMETRIC` records are emitted only by the node that is observed as the PostgreSQL leader, avoiding duplicate HA counts. If a gateway management observation becomes unavailable, the cockpit shows **Management observation unavailable** and **Not observed** for unavailable gateway metrics; this means the management path could not be observed and does not prove the gateway or LoRaWAN radio path is offline. Sensor telemetry has separate semantics: a successful database read with zero rows is `idle` / **No fresh packets**; a failed live query is **Telemetry observation unavailable**; a transient read failure with a previous valid snapshot retains that snapshot as `stale`; only a read failure with no usable snapshot is an error. The clean fresh-test baseline is therefore expected to show **No fresh packets** until real new AS923 uplinks arrive. Telemetry queries use bounded five-second reads and a 30-second cache horizon, and their SSH subprocesses explicitly use a closed stdin (`DEVNULL`) so a detached/managed cockpit worker cannot block waiting on inherited console input. These rules prevent a slow polling cycle or unavailable management path from being mislabeled as device failure.

Recorder v4 adds a **foreground supervisor**. `run` is now the preferred Chapter 4 entry point: it starts the same serial/remote collectors as `start`, performs a bounded collector/evidence health gate every 15 seconds by default, records those observations in `raw/control-plane-health.csv`, and invokes the normal `export/summarize/hash` finalizer automatically on duration completion or `Ctrl+C`. This prevents an unnoticed workstation-session loss from leaving stale active state and silently invalidating later trials. Each supervisor row records the server-authoritative UTC timestamp, the workstation midpoint timestamp around that clock probe, the measured server-minus-workstation offset, and probe round-trip time. This makes workstation clock skew explicit instead of silently mixing clock domains. Experiment timing still uses server-authoritative timestamps and source/database evidence as described below.

Recorder v5 hardens that supervisor for real multi-minute counted work. A dead collector process remains an immediate terminal failure because evidence is actually being lost. Short-lived remote-readiness, USB/re-enumeration, database-role, or clock-probe failures are instead retained as `WARN` observations and must persist for three consecutive health gates before the supervisor aborts; a later clean gate resets the transient-failure count. Per-node SSH/readiness timeouts are converted into explicit health details rather than uncaught exceptions. If the server-authoritative clock probe is temporarily unavailable, the CSV leaves the authoritative timestamp and skew fields blank, records the condition as a warning, and never substitutes workstation time as if it were server time. `status`, `stop`, and `recover` also tolerate a failed final health probe so preservation/finalization can still complete. This policy was added after the 2026-09-04 uncounted counted-profile reload capture showed that a single transient readiness timeout could terminate the foreground supervisor while all child collectors and the actual LoRaWAN path remained healthy.

Workstation dependency

The recorder is Python-native because this workstation is governed by a domain PowerShell `MachinePolicy=Restricted`. Do not weaken that policy.

Python dependency is pinned in `requirements.txt`:

```
pyserial==3.5
```

Install/reproduce with:

```
python -m pip install --user -r .\lorawan-network-server-gateway\test\automation\research-recorder\requirements.txt
```

The older `.ps1` prototypes in this directory are not the operator entry point on this workstation.

Locked-down SSH boundary

The recorder uses a dedicated key outside the repository:

```
%USERPROFILE%\ssh\id_ed25519_research_recorder
```

The private key is never stored in the project. Its public key is installed on the three cloud nodes and Gateway-01 with a forced-command wrapper. The key can invoke only the `read/capture` verbs implemented under `remote/`; an arbitrary command such as `uname -a` is rejected.

Never replace this with an embedded SSH password or copy an administrator private key into the repository.

Forced-command monitor boundary - current state 2026-09-18

Fresh restricted-key version checks on ulc-01, ulc-02, and ulc-03 all return research-recorder-server-v7, matching the current repository wrapper. Do not reuse the historical v5 wrapper hash as a v7 integrity claim; verify the deployed file hash separately whenever byte-for-byte wrapper attestation is required. Gateway-01 remains on its separately versioned restricted gateway wrapper. The dedicated recorder key remains forced to reviewed verbs; arbitrary shell commands are not permitted.

Server wrapper v7 retains the bounded recorder/readiness verbs, `monitor-stream <seconds>`, and the fixed read-only `db-export packetflow <startUTC> <endUTC> <gatewayEUI>` projection used by the live packet correlator. The monitor stream multiplexes one-second host-resource records with slower service, readiness, database-role, database-counter and bounded recent-error observations. Optional service/error side probes are deliberately non-fatal, so an empty error scan or one failed auxiliary observation cannot terminate the resource stream and silently leave the cockpit stale. The packetflow projection joins already-authoritative gateway MQTT evidence to accepted telemetry, evidence-verification status, Fabric outbox state and Fabric transaction fields. It accepts no arbitrary SQL, table name, column name or filter expression from the caller.

Gateway wrapper v2 retains snapshot, resource and logstream and adds bounded `radio-recent <count>` and `journal-recent <count>` reads. `radio-recent` exposes only recent RAK5146 frame-reception log lines. `journal-recent` exposes retained immutable gateway journal records needed to correlate exact PHYPayload bytes for authorized replay testing. Neither command changes radio, network or journal state.

The version verbs and the fixed packetflow/radio reads were verified through the same restricted recorder identity after deployment. Keep the repository wrappers and deployed copies converged; do not bypass or weaken the forced-command boundary to troubleshoot monitoring.

Read-only evidence-status boundary

The bounded evidence-status capability remains part of the current wrapper. It executes fixed SELECT-only queries against the commissioned evidence status views and returns aggregate verification/checkpoint state only. It exposes no payload bytes, evidence hashes, database credentials, arbitrary SQL, or arbitrary shell access.

The older `UNAVAILABLE_WRAPPER_UPGRADE_REQUIRED` compatibility behavior is historical only. Current cloud nodes run wrapper v7, so new formal runs are expected to capture the fixed evidence-status output when the underlying evidence read path is healthy. If a current run receives `command not allowed` or an `unavailable-wrapper` marker from a cloud node, treat that as a deployment/version regression and investigate it rather than accepting it as the present baseline.

Journal-read prerequisite

opsadmin must be a member of `systemd-journal` on ulc-01, ulc-02, and ulc-03. This is required so the forced-command recorder identity can retain Mosquitto and kernel-journal evidence without using `sudo` or an unrestricted administrator shell. The membership was applied and verified on all three cloud nodes on 2026-09-03.

Accepted recorder smoke - 2026-09-03

The first uncounted smoke, `recorder-smoke-20260903`, proved the core `start/capture/stop/export/summarize/hash` path but exposed the missing journal-read permission above; preserve that run as diagnostic evidence. After correcting journal access, `recorder-smoke-journal-20260903` completed cleanly with all eight live collectors healthy, recorder stop exit 0, status `RECORDED_UNCLASSIFIED`, no stop-error file, 65 SHA-256 manifest entries independently verified, and 39 raw evidence files sealed read-only. The marker path, TimescaleDB exports, exact-window service logs, kernel logs, statistics, and active-state cleanup were also verified. Static/live recorder validation then passed.

Both smoke runs intentionally used `--skip-serial`. Counted LoRaWAN runs must pass `preflight --require-emu --require-sec` and must not skip EMU-01 source capture when the PDR denominator depends on `SENSOR_TX`

attempts.

Accepted supervised-recorder smoke - 2026-09-04

Recorder v4 was commissioned with two bounded, uncounted foreground run `--skip-serial` smokes against the live Gateway-01 plus three-node cloud. The first run, `collector-supervised-smoke-20260904-0456z`, proved collector supervision and clean automatic finalization but deliberately remains diagnostic evidence because it exposed that its first implementation timestamped supervisor health with workstation UTC while start/end boundaries used server-authoritative UTC. The workstation was materially behind the cloud clock, so those control-plane rows could appear before the run start. The same run also showed that uncounted metadata must not claim the frozen counted EMU profile when serial capture is skipped.

Both provenance defects were corrected before acceptance. `control-plane-health.csv` now records server-authoritative UTC, the bracketing workstation midpoint, measured server-minus-workstation offset, and clock-probe round-trip time. `--skip-serial` metadata now records `serial_capture_enabled=false`, `emu_profile=NOT_CAPTURED_UNCOUNTED`, and leaves counted source/HEX hashes blank rather than implying the counted firmware was observed.

The corrected run `chapter4-results/smoke/collector-supervised-clockfix-20260904` is the accepted v4 supervisor smoke. It exited 0, finalized as `RECORDED_UNCLASSIFIED`, produced 76 manifest entries with zero missing files and zero SHA-256/size mismatches, left no `_recorder-active.json`, `stop-errors.txt`, or `supervisor-errors.txt`, and recorded three monotonically increasing `PASS` supervisor rows inside the authoritative start/end window. The clock probes measured the server about 104.65-104.90 s ahead of the workstation with roughly 1.06-1.22 s round-trip time; this skew is now explicit evidence rather than a hidden timing assumption. A subsequent live preflight `--require-emu --require-sec` also passed with EMU-01 on COM11, SEC on COM16, ulc-01 leader, ulc-02/ulc-03 replicas, and all required evidence roles ready.

Counted-profile post-flash sanity - 2026-09-03

The uncounted readiness run `counted-profile-postflash-sanity-20260903` exercised the frozen 15-second EMU-01 image with real USB source capture and the commissioned Gateway-01/LTE/cloud path. The sealed recorder-v1 bundle contains 189 healthy source attempts (`send_status=0`, sequences 7..195), 192 unique database event keys, 2,496 normalized measurements (exactly 13 per event), and 192 Fabric-outbox rows left pending because the external Fabric worker remains intentionally disabled.

A naïve comparison of 192 DB rows against 189 captured source attempts produced an impossible 101.587% value in the sealed v1 derived summary. That number is **not a valid PDR and must not be cited**. Byte-for-byte reconstruction of each 46-byte payload proved 185 exact source/DB matches for sequences 7..191. Those matches establish a stable DB-minus-workstation timestamp offset of about +103.24 seconds. After applying that measured offset to define the source attempts actually covered by the DB export window, all 185 eligible source attempts have exact DB matches: aligned PDR = 100.0%. DB-only rows are the six current-session frames emitted before serial capture plus one prior-session sequence-8 frame admitted by the skewed wall-clock start boundary; source-only sequences 192..195 project beyond the DB export end.

Recorder version 2 therefore no longer calculates PDR as `DB-row-count/source-attempt-count`. It reconstructs the payload-v2 bytes from every `SENSOR_TX` line, correlates them exactly to `raw_data`, estimates the workstation/cloud clock offset from those exact matches, and limits the denominator to source attempts whose projected DB timestamps fall inside the actual export window. If that aligned observation window cannot be established, PDR is withheld rather than guessed. The sealed v1 bundle is intentionally left unchanged; corrected interpretation is recorded in the preflight analysis outside the sealed run.

The same sanity run exposed a Windows-specific shutdown defect: `taskkill /T /F` could hang while traversing the process tree. Direct `taskkill /PID <pid> /F` was proven against the live recorder-owned processes and a disposable child. Both Python and legacy PowerShell recorder paths now omit `/T`; the Python path also bounds termination to five seconds.

Result shape

```
chapter4-results/<group>/<run-id>/
raw/
  emu-01-source.log
  sec-source.log           # when requested
  gateway.log
  gateway-resource.csv
  ulc01-host-resource.csv
  ulc01-docker-resource.csv
  ulc02-host-resource.csv
  ulc02-docker-resource.csv
  ulc03-host-resource.csv
  ulc03-docker-resource.csv
  uplinks.csv
  measurements.csv
  fabric-outbox.csv
  *-chirpstack.log
  *-mosquitto.log
  *-node-red.log
  *-kernel.log
  markers.csv
metadata/
  run-meta.json
  start-utc.txt
  end-utc.txt
  *-snapshot-start.txt
  *-snapshot-end.txt
  database-roles-end.json
  SHA256SUMS.csv
derived/
  run-summary.json
  run-summary.csv
  trial-results.csv
  run-status.txt
```

PASS, FAIL, INVALID, and BLOCKED remain research classifications. The recorder deliberately ends a clean capture as RECORDED_UNCLASSIFIED; evidence collection must not invent the experiment outcome.

Usage

From the repository root:

```
$R = './lorawan-network-server-gateway/test/automation/research-recorder/research_recorder.py'

# Remote collectors + Patroni topology; add both flags before LoRaWAN security testing.
python $R preflight --require-emu --require-sec

# Preferred counted-run mode. Keep this terminal open; Ctrl+C finalizes safely.
# Use a second terminal for mark/trial/serial-control commands while this runs.
python $R run `
  --group authentication `
  --run-id auth-lorawan-A1-01 `
  --condition 'registered device with correct OTAA credentials' `
  --expected 'allow; JoinAccept and accepted uplink' `
  --scope lorawan `
  --capture-sec

# Timestamp a physical/software event without stopping capture.
python $R mark --message 'A1-01 reset issued'

# Read the current aggregate verifier/checkpoint state without starting a run.
# This is intentionally strict: on pre-v3 live wrappers it reports that the server-side upgrade is required.
python $R evidence-status

# Optional explicit active-recorder view; the foreground supervisor already gates health every 15 seconds.
python $R status
```

```

# When a counted OTAA trial requires reloading the exact frozen EMU image,
# keep the recorder and all remote collectors alive while intentionally yielding COM11.
# The serial log records RECORDER_SERIAL_PAUSED / RECORDER_SERIAL_RESUMED and then
# automatically reacquires the board after DFU re-enumeration.
python $R serial-pause --device emu
# reload the already-frozen counted-test ZIP; do not rebuild here
python $R serial-resume --device emu

# The same mechanism may yield SEC during A2/A3 fixture commands without stopping
# the rest of the counted evidence capture:
# python $R serial-pause --device sec
# ... issue only the documented SEC test-fixture commands ...
# python $R serial-resume --device sec

# Record the trial classification after its evidence is inspected.
python $R trial `
  --trial-id A1-01 `
  --condition 'correct OTAA credentials' `
  --expected allow `
  --actual allow `
  --status PASS `
  --notes 'JoinRequest received; JoinAccept; application uplink stored'

# Normally press Ctrl+C in the foreground `run` terminal when complete.
# It automatically exports DB/log evidence, calculates statistics, hashes files and seals raw evidence read-only.

# Recovery only: if a detached or interrupted run left _recorder-active.json,
# inspect it first, then finalize and preserve it rather than deleting it.
python $R status
python $R recover

```

For a bounded **uncounted** collector smoke test, run `--skip-serial --duration-seconds 60` is appropriate. In that mode `run-meta.json` explicitly marks serial capture disabled and the EMU profile as `NOT_CAPTURED_UNCOUNTED`; it does not claim that the frozen counted-test image was observed. Counted runs normally omit `--duration-seconds` and run until operator Ctrl+C.

Use `--interval 1` for flooding and normally `--interval 5` for baseline/resilience/resource observations. The default supervisor health interval is 15 seconds; do not make it aggressively shorter because each gate also checks the replicated evidence path.

`--skip-serial` exists only for clearly uncounted recorder smoke tests. Do not use it for a counted run whose delivery/PDR denominator depends on EMU-01 `SENSOR_TX` attempts.

Statistical interpretation

The recorder derives only metrics supported by its actual evidence. Formal Chapter 3 PDR and source-to-database delivery are deliberately separate:

```

formal Chapter 3 PDR = unique accepted EMU-01 ChirpStack deduplication IDs across both HA ChirpStack logs /
captured scheduled SENSOR_TX attempts x 100
database delivery    = exact DB payload matches for eligible source attempts / eligible source attempts x 100

```

For formal PDR, `summarize-run.py` strips ANSI log formatting, unions accepted `deduplication_ids` from `ulc01-chirpstack.log` and `ulc02-chirpstack.log`, associates them with the run's `dev_eui` from `run-meta.json`, and divides that unique accepted count by every captured scheduled `SENSOR_TX` attempt in the same run window. A scheduled source-side send failure remains in the denominator. The percentage is withheld when required logs/device identity/source attempts are missing or when accepted uplinks exceed the source-attempt boundary. This is the Chapter 3 LoRaWAN/ChirpStack metric boundary.

For database delivery, source and database evidence are correlated by reconstructing the exact 46-byte payload-v2 bytes from each `SENSOR_TX` line and matching those bytes to `telemetry.uplinks.raw_data`. Exact matches also provide the measured DB-minus-source clock offset. A source-side attempt remains in the delivery-correlation denominator only when the recorder can prove that attempt lies inside the DB observation window. Boundary attempts outside that comparable window are reported as unmatched correlation evidence, not silently counted as packet loss.

Never copy database delivery into a Chapter IV PDR column merely because both percentages may happen to be equal in a healthy run. An MQTT redelivery is not a second LoRaWAN delivery, and a second LoRaWAN session reusing a sequence/FCnt value is not collapsed merely because the counter repeats.

For replay/spoofing, only attack transmissions whose RAK5146 reception is proven belong in the counted attack denominator. `sec02_replay.py capture --test-sequence <N>` may now bind a replay/spoofing fixture to the exact accepted EMU-01 `test_sequence` designated as that trial's legitimate control; the helper still requires the configured minimum FCnt age and independently correlates the selected packet to Gateway-01 journal, packetflow, and RAK5146 reception evidence before it will create the fixture. This prevents a nearby historical uplink from being silently substituted for the intended control.

`application_event_to_db_ms` is `telemetry.uplinks.received_at - telemetry.uplinks.time`. Do not relabel it sensor-to-database latency unless a separate run proves the source/laptop UTC timestamp is correlated closely enough with the server clock.

The post-run summarizer also reports sample SD, min, max, RSSI, SNR, frequency counts, sequence/FCnt gaps, measurement counts, Fabric outbox states, and per-node host/container resource statistics.

What remains physical

Evidence capture, exports, statistics, hashing, most attack generators, and server/network fault injection can be automated. Human action remains only where the experimental condition itself is physical, such as reconnecting actually disconnected hardware, deliberately moving nodes/antennas, or changing a real environmental sensor condition.

A physical reset can be timestamped with `mark`. If a test later proves a safe software-reset mechanism for the exact board mode, a test-specific driver can automate that reset as well.

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Research Manual Support Tooling

The authoritative operator-facing test manual is:

```
chapters/lorawan_research_test_manual_final.pdf
chapters/lorawan_research_test_manual_final.md
```

The manual is aligned to `test/ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md`. It keeps the Chapter 3 experiment counts, durations, formulas, evidence boundaries, validity rules, and measurements, but deliberately omits stale implementation prose.

`CONTINUATION.md` is the current handoff for future chats. Read it before changing the manual or test harness.

To exercise the **same operator dispatch without live hardware or production effects**, use `py -3 test/automation/research-manual/run_test.py I3 --synthetic` from the repository root, replacing `I3` with `A1`, `A2`, `P2`, `R1`, `R2`, `S1`, `F1`, `F2`, `I1`, `I2`, `I3`, `T1` or `T2`. This runs existing allowlisted synthetic tests, never starts a counted run, and does not promote live readiness. `PRE/P1` do not have complete offline equivalents; `S2` is methodology-blocked. See `SYNTHETIC-COMMAND-REHEARSAL.md` for the checked 13-ID command table and limitations.

`run_test.py` is the fail-closed operator/readiness entry point. A code block may appear in the final manual only when the corresponding test is `READY` and the complete action is runnable. Non-ready tests remain documented without a code block until their harness is commissioned.

Status meanings remain:

- `READY` - the existing harness can perform the complete formal action when preflight passes.
- `CAPTURE_READY` - evidence capture exists, but the deliberate condition/fixture action still has to be performed and proven at the tested layer.
- `HARNESS_REQUIRED` - Chapter 3 is specified but a reviewed fixture/orchestrator is still required; counted execution must fail closed.
- `METHODOLOGY_REQUIRED` - Chapter 3 itself is incomplete for that experiment; counted execution must remain blocked until formally amended.

A green infrastructure preflight is separate from formal experiment readiness. `PRE` proves the commissioned platform, clocks, sensor/gateway path, evidence services, and access paths are healthy. It does not relabel incomplete experiment harnesses as ready.

`CODE-BLOCK-VALIDATION.md` records only released operator-block validation. `CURRENT-READINESS.md` is a point-in-time implementation snapshot, not a substitute for the manual or Chapter 3 source-of-truth.

Verify a sealed run without changing its evidence

From the project root:

```
python .\test\automation\research-recorder\verify-run-reproducibility.py .\chapter4-results\smoke\RUN_ID
```

Replace RUN_ID with a completed run directory. The checker validates every manifest entry and requires all raw files to be sealed. It regenerates summaries twice in a temporary copy. JSON measurements must match the sealed summary exactly (only the relocated run_dir is excluded); CSV output must be byte-identical. A mismatch exits nonzero. Original evidence is never rewritten.

This proves reproducible processing of the recorded inputs with the reported Python version and summarizer SHA-256. It does not establish statistical repeatability of radio measurements or validate an uncommissioned experiment.

Readiness reporting distinguishes tooling from live checks. With --skip-live-pre, a successful local check reports tooling_gate=PASS and technical_gate=SKIPPED. Only successful live checks can report technical_gate=PASS. Code-block validation checks the interface; it is not evidence that the experiment itself ran.

Supporting runbook: CONTINUATION.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Research Test Manual Continuation

Gateway admin access investigation — 2026-09-21 (~05:55 UTC; latest)

Current Step: Cloud MQTT bridge still has zero established sockets; only authorized restricted research-recorder SSH works. Cloud opsadmin password is not Gateway-01 root password. User granted broad SSH operational authority; credentials are still a separate technical requirement.

Completed: Rechecked live gateway LTE `wwan0=100.73.133.167/28`, registered and public broker/default routes on LTE, no `:8883` sockets. Workstation full SSH inventory, Ubuntu WSL SSH identities, Windows Credential Manager and ULC-01 operator/protected key-name search yielded no alternate Gateway-01 administrator identity. Gateway SSH port is up; LuCI port 443 is up but replies `403 / x-luci-login-required`. The current research-recorder SSH identity remains forced-command diagnostic-only; attempted gateway root password login with existing opsadmin secret failed and was stopped without guessing. Other production infrastructure left unchanged. Full gateway-root DNS/TCP/mosquitto diagnosis and repair remain blocked by unavailable gateway administrator authentication. Detailed findings and recovery checks: `test/automation/research-manual/F2-PARTIAL-REHEARSAL-AND-GATEWAY-RECOVERY-20260921.md`.

Next Step: Obtain the actual Gateway-01 administrative root credential or an authorized admin private key/session, then inspect `nslookup smartagri-mqtt.duckdns.org, /etc/resolv.conf`, LTE-bound TCP `:8883`, `logread -e mosquitto`, cert validity and mTLS health. Fix only evidenced root cause, prove two `:8883` LTE bridge sockets and a fresh EMU-01 Fabric-confirmed uplink, then run full non-counted F1/F2. Do not weaken authentication or route sensor cloud traffic over management Ethernet.

Gateway bridge recovery boundary and F2 isolated actor — 2026-09-21 (latest)

Current Step: F1/F2 end-to-end live testing stays blocked because Gateway-01 has no established MQTT `:8883` uplink/downlink bridge. `wwan0`, LTE default route and local AS923 EMU-01 frame reception remain up; ULC-01's DNS resolves the public MQTT hostname, Windows can connect to `129.212.208.168:8883`, but no fresh cloud EMU uplink. Gateway Mosquitto repeatedly logs `Error creating bridge: Try again`. Exact failing gateway DNS/LTE TCP/TLS layer is not proven without admin diagnostics. Available restricted gateway SSH key cannot run those; the known cloud opsadmin password failed for Gateway-01 root. No production config, routes or mTLS changed. Do not bypass the gate.

Completed: Actual isolated F2 Node-RED + MQTT malformed-message path with existing restricted research fixtures: 0/0 (0/s 3s), 40/40 (10/s 4s), 200/200 (50/s 4s) messages generated/published/received/rejected, invalid accepted 0. The all-in-one subprocess hit a global timeout **after tunnel cleanup**; independent final inspection proved Node-RED research nodes absent, original flow hash restored, healthy container, isolated listener INACTIVE. F2 synthetic test 16/16 PASS, F1 isolated real conn-flood actor also passed in previous run. Neither is a complete live flood research trial, counted=0. Evidence and admin-only recovery checklist: `test/automation/research-manual/F2-PARTIAL-REHEARSAL-AND-GATEWAY-RECOVERY-20260921.md`.

Next Step: Establish authorized Gateway-01 admin SSH access and verify resolver + LTE-bound TCP :8883 + bridge-specific logs/cert before targeted recovery. Confirm new EMU-01 DB uplink and Fabric-confirmed outbox, then run full non-counted F1/F2 via manual command with true 0/10/50 per-window sensor-delivery checks. Do not rerun isolated actors or full 198/198 regression without a pertinent code change.

F1 partial live flood commissioning / gateway cloud-bridge outage — 2026-09-21 (current)

Current Step: End-to-end F1 is blocked by a real gateway-to-cloud MQTT outage; do not use the isolated flood test as Chapter 4 PASS.

Completed: The real manual `run_test.py` F1 rehearsal failed closed at technical preflight: `GATEWAY_LTE=FAIL` solely because gateway MQTT :8883 bridge has no established session; LTE `wwan0` and default/public broker route remain up and registered, live PRE skipped. Gateway concentrator/forwarder continue receiving/sending local AS923 EMU frames every ~15s, but bounded cloud DB export found no new uplinks. Gateway logs show repeated cloud-uplink/cloud-downlink bridge creation errors Try again to `smartagri-mqtt.duckdns.org:8883`; cloud resolves its intended Reserved IP and public :8883 is reachable from workstation. Root cause within gateway LTE DNS/network/bridge is NOT independently isolated; no privileged gateway change or bypass was made. F1 synthetic code 16/16 PASS; genuine **isolated flood-only actor** also PASS: 0/0, 40/40 (10/s for 4s), 200/200 (50/s for 4s) broker-observed/rejected, invalid accepted 0, local errors 0, tunnel/listener cleanup PASS. This does not test legitimate sensor delivery during flood windows. `test/automation/research-manual/F1-PARTIAL-REHEARSAL-20260921.md` has the exact fault boundary, evidence and recovery checks. Formal research counted=0.

Next Step: Restore Gateway-01's two mTLS MQTT bridges over LTE (not LAN) through an authorized admin channel, then verify fresh ChirpStack DB/Fabric flow and rerun the actual F1 non-counted 0/10/50 windows. Do not run F2/other live tests requiring cloud telemetry while this gate is failing.

A1 live LoRaWAN + isolated MQTT commissioning — 2026-09-21 (latest)

Current Step: A1 LoRaWAN and MQTT real non-counted condition sets have passed. The separate Fabric authentication third and all formally counted nine-condition x10 trials remain unreleased.

Completed: Exact `a1_lorawan_harness.py --rehearsal` exit 0: **3/3 real LoRaWAN conditions correct**, gateway JoinRequest journal sequences 3864/3865/3866 bound to correct JoinEUI/DevEUI, wrong AppKey rejected without altering registered ChirpStack session, unregistered DevEUI rejected with registered=0, legitimate EMU OTAA allowed with new session and successful uplink; SEC parked and EMU same SHA-256 firmware restored. Evidence `chapter4-results/authentication/lorawan/A1-LORAWAN-rehearsal-20260921-131919/`. First MQTT rehearsal failed closed due stale six-field live helper vs repo seven-field observer-proof parser; preserved failed report, updated **only isolated ULC-01 helper** after backup and hash check to repo SHA-256 `cb0487fdc21f29b6aa9c002e0c1d1b98dd0ec197777ad8f454f2ffd7b79965db`. Second `a1_mqtt_harness.py --rehearsal` exit 0: **30/30** isolated MQTT decisions: 10 allowed observed, 10 wrong-password rejected/unobserved, 10 prohibited-topic rejected/unobserved, false acceptance/rejection 0, unauthorized delivery 0, isolated listener stopped and independently status INACTIVE. Evidence `chapter4-results/authentication/mqtt/A1-MQTT-rehearsal-20260921-132300/`. Updated A1 manual status wording, not status: still `CAPTURE_READY` until Fabric-side real authentication actor and synchronized methodology commissioned. Full detail `test/automation/research-manual/A1-LIVE-REHEARSAL-20260921.md`. Non-counted; offline baseline **198/198** remains.

Next Step: Safely commission one isolated real F1/F2 flood window with legitimate EMU delivery (prefer harness built-in bounded rehearsal), or commission A1 Fabric-side fixture when HRC actor is available. Do not re-run already proved P1/T1/T2 or 30-case MQTT without a pertinent change.

T2 live history/chronology commissioning — 2026-09-21 (latest)

Current Step: The exact T2 manual operator command now has a successful non-counted five-record real-world rehearsal. Formal Chapter 4 trial count remains zero.

Completed: RESEARCH_MANUAL_REHEARSAL_SECONDS=120 and run_test.py T2 with EXECUTE exited 0, TRACEABILITY_HARNESS=PASS test=T2 formal=False. Closed 120-second window contained **8/8 confirmed uplinks/outbox rows**, no missing source events, and after-window Fabric settlement completed. Selected independent database-leader readback: **5/5 original event keys retrieved** with frame counters **169–173**, test sequences **170–174**, 13 distinct normalized measurement fields per reading, 5/5 confirmed outbox transaction IDs and SHA-256 digests, chronological-order accuracy **100%**, missing/duplicates **0/0**. Each of the five USB-powered records correctly carries battery_v=NULL quality=invalid; no voltage is fabricated. The five-record reconstruction took **~4.458 s**. Evidence: chapter4-results/traceability/T2-rehearsal-20260921-130551/ and chapter4-results/traceability/_sessions/T2-20260921-130551.json; explanation: test/automation/research-manual/T2-LIVE-REHEARSAL-20260921.md. Live PRE, LTE, clock and recorder health passed. No counted trials.

Next Step: Traceability T1/T2 capture/retrieval code is live-rehearsed, but formal release still needs independently observed HRC ledger query results (current scorer verifies confirmed DB outbox+TxID) and full formal sampling. Choose the next isolated, non-counted intervention test (A1 LoRaWAN/MQTT, F1/F2 or R2), inspect its exact restricted-action readiness and perform one bounded genuine trial. Avoid repeating P1/T1/T2 without code or infrastructure change.

T1 live traceability commissioning — 2026-09-21 (latest)

Current Step: One full non-counted T1 capture and independent per-record DB retrieval has passed. Do not count any of these as Chapter 4 repetitions.

Completed: The existing sealed P1 run initially exposed a real T1 scorer failure: USB-powered EMU-01 stores battery_v=NULL, quality=invalid, source_field=battery_v rather than a fabricated 0 V reading. Corrected the scorer to preserve exactly this known-unavailable field as value:null while rejecting unrelated missing measurements; added 2 targeted regressions (11/11 T1/T2 unit tests). Replayed 2/2 genuine P1 records against fresh independent DB leader readbacks and outbox-confirmed Fabric TxIDs: PASS. Then executed **the exact manual T1 operator command** with RESEARCH_MANUAL_REHEARSAL_SECONDS=55: exit 0; TRACEABILITY_HARNESS=PASS formal=False; closed-window 3/3 source events and outbox transactions confirmed; 2/2 independently re-retrieved selected records, each 13 fields (12 available values and explicitly invalid/unavailable battery), source ID, timestamp, frame counter, digest, and TxID present; no missing/duplicate selected records. SHA-256 sealed run chapter4-results/traceability/T1-rehearsal-20260921-125602/; separate scorer JSON chapter4-results/traceability/_sessions/T1-20260921-125602.json. No independent HRC ledger query by this scorer (it verifies confirmed database outbox with TxID); do not promote formal T1 to READY on that basis. Latest consolidated code-only suite **198/198 PASS across 16 suites**, chapter4-results/code-qualification/offline-code-20260921T050059692509Z/code-only-summary.json. Full scope and field-availability explanation: test/automation/research-manual/T1-LIVE-REHEARSAL-20260921.md.

Next Step: Commission T2 non-counted five-reading chronology with its own fresh run, then resolve independent HRC ledger query evidence before releasing formal traceability operator blocks. Check live evidence schema v2 versus frozen I2 v1 helper when I2 is commissioned.

Rapid hardware-connected readiness sweep — 2026-09-21 (current)

Current Step: The PRE + bounded P1 operator path and all available synthetic operator commands have passed; formal research runs remain uncounted. No other live experiments were initiated by this sweep.

Completed: First P1 launch correctly failed closed because oversight_status.py checked the obsolete source substring query < status < retry, whereas the strengthened R2 harness checks query < verify < status < retry (Query → VerifyDigest → CommitStatus → same-prepared-transaction retry). Corrected only that stale audit token; OVERSIGHT_AUDIT=PASS, TECHNICAL_GATE=PASS, LIVE_PRE=PASS, and GATEWAY_LTE=PASS. Executed **the exact P1 manual entry point** with RESEARCH_MANUAL_REHEARSAL_SECONDS=65 and EXECUTE, using a non-counted smoke run. It exited 0 and reported P1_MEASUREMENT_BOUNDARY=PASS: **4 source attempts; 4 ChirpStack-accepted; 4 database uplinks; 4/4 Fabric-confirmed with transaction IDs**. The after-window Fabric settlement reached CONFIRMED (4/4), collector health checks passed, clock-calibration drift was 0.001193 s (< 0.100 s limit), and the recorder marker was removed. Saved run chapter4-results/smoke/manual-P1-rehearsal-20260921-124351/ contains derived/run-summary.json, metadata/fabric-settle.json, source/measurement/outbox CSVs and SHA256SUMS. RECORDED_UNCLASSIFIED is the normal non-counted run status, not a formal PASS classification. Executed **13/13** available run_test.py <ID> --synthetic commands successfully (A1 A2 P2 R1 R2 S1 F1 F2 I1 I2 I3 T1 T2). PRE, P1 and S2 correctly report SYNTHETIC=UNAVAILABLE, not a fake PASS. Prior full code regression is **196/196 PASS**, and only the oversight guard changed in this sweep.

Next Step: The hardware and real P1 measurement/observation pipeline are usable. For each other formal test, commission its own attack/outage/Fabric actor and real evidence path before promoting it to READY; avoid equating synthetic code PASS with live experiment PASS. S2 remains METHODOLOGY_REQUIRED. No counted Chapter 4 repetitions have been started.

Live hardware reconnection — 2026-09-21 (current)

Current Step: Real full-stack smoke is verified, not a counted Chapter 4 experiment. The prior hardware-disconnected instructions below are historical. See test/automation/research-manual/REAL-HARDWARE-RECONNECT-20260921.md for precise evidence and source paths.

Completed: Workstation Ethernet 192.168.20.30/24 to Gateway-01 192.168.20.11; EMU-01 COM11, SEC-01 COM16. Real recorder preflight --require-emu --require-sec exited 0: ULC-01/02/03 and gateway reachable, one database leader/two replicas, cloud evidence services ready, calibrated CLOCK_GATE PASS. Gateway LTE wlan0=100.73.133.167/28 retained the default/cloud MQTT route; management LAN separate. Corrected RUI3 query-echo bug in A1 LoRaWAN harness and added regression (targeted 16/16 PASS); one controlled unregistered SEC join produced fresh 923.2 MHz SF10 gateway frames and SEC was re-parked. EMU-01 was restored with its SHA-256-verified 15-second research firmware via successful COM12 DFU. Retained serial showed OTAA JOIN PASS and three successful source sends at 15007/15006 ms intervals, valid=0x7F. Initial canary summary remains FAIL because boot-only banner was missed; corrected canary accepts the archived verified DFU plus measured cadence as alternative profile proof, without rewriting the old report. Real DB readbacks: five new EMU uplinks and 65 measurements (13 each); subsequently gateway packetflow showed fifteen EMU records, nine evidence-verified and Fabric-confirmed with transaction IDs, six newer records pending asynchronous settlement at observation time. evidence_gap=0, integrity_failure=0. Representative Fabric TxID 042c46dbcbf88f79ae85e60c7acfa78423a4573a09e29b37b0575b7ecef69bd. No synthetic or real rehearsal records were counted as formal research samples.

Next Step: Once ready to execute a research operator test, run a bounded non-counted P1 recorder rehearsal and inspect its closed-window source/database/Fabric reconciliation. Do not promote uncommissioned A1/F1/F2/R1/R2/T1/T2/I1/I2/I3 tests to READY based on this one smoke. S2 still needs methodology. The 15-second EMU research firmware remains installed; avoid leaving unnecessary continuous sample traffic when hardware is idle. The complete offline suite was rerun after the live-derived A1 parser fix: **196/196 PASS across 16 suites**, non-counted report chapter4-results/code-qualification/offline-code-20260921T043222317342Z/code-only-summary.json.

Streamlined synthetic operator-command rehearsal — 2026-09-21

Current Step: Keep the gateway/sensors disconnected and verify real operator command dispatch with synthetic fixtures, not counted measurements. `run_test.py <ID> --synthetic` now uses `offline_test.py` before any live preflight, while ordinary `run_test.py <ID>` retains its existing guarded/manual execution path.

Completed: All **13/13** mapped test IDs (A1 A2 P2 R1 R2 S1 F1 F2 I1 I2 I3 T1 T2) passed their exact operator --synthetic command. PRE P1 S2 correctly returned `SYNTHETIC=UNAVAILABLE` (nonzero), not fake PASS. Ordinary `run_test.py I3` still reports `HARNESS_REQUIRED` and does not start counted work. Static Python compile and diff checks passed. See `test/automation/research-manual/SYNTHETIC-COMMAND-REHEARSAL.md`. The preceding full consolidated suite remains **195/195 PASS** across 16 suites; the dispatch mapping adds no new counted trials.

Next Step: Once physical hardware returns, run real PRE and one genuine sensor-to-Fabric smoke flow. Then commission unready experiments against their actual production paths one by one; do not mark synthetic samples as live. S2 requires a methodology definition before a valid experiment can exist.

Latest checkpoint — 2026-09-21 hardware-offline test-code batch

Current Step: Stop all tests requiring Gateway-01 or powered sensors. Qualify existing research test code one harness at a time with deterministic, non-counted fixtures. Earlier live infrastructure notes below are historical, not a claim about currently disconnected hardware.

Completed: One-command offline qualification now passes **196/196 tests** across research contract, A1 MQTT, A1 LoRaWAN, A2 Fabric endorsement-evidence scoring, I1 experimental application-integrity gate/scorer, I2 read-only production-v1 source recomputation/evidence scorer, I3 source-bound ledger duplicate/overwrite evidence scorer, Fabric synchronization, P2 pair runner, R1 LTE route gate, R2 reconciliation, F1/F2 flooding validation, T1/T2 traceability, S1 replay helper, and mock MQTT wire. The exact machine-readable output is `chapter4-results/code-qualification/offline-code-20260921T043222317342Z/code-only-summary.json`; details and one command to rerun are in `test/automation/OFFLINE-CODE-QUALIFICATION.md`. New guards prevent stale JoinRequests, wrong JoinEUI matches, duplicated outage event keys, arbitrary P2 resume PASS summaries, incomplete 13-metric readings, and chronology-score false positives. No gateway/sensor, production MQTT, or live Fabric mutation was invoked in this batch.

A1-specific outcome: `py -3 test/automation/offline_test.py A1` returned `CODE_ONLY_PASS` for 30/30 non-counted checks (contract 6, MQTT 6, LoRaWAN packet/state 15, LoRaWAN mocked workflow 3). The LoRaWAN guards now reject stale or wrong-JoinEUI packets, missing/changed server evidence and valid-join claims without verified new session; mocked cleanup failure forces FAIL. `join_observation_ms` measures the local join-observation interval; it is not a server processing-time measurement. The newly edited helpers have not been live-commissioned.

P2-specific outcome: `py -3 test/automation/offline_test.py P2` returned `CODE_ONLY_PASS` for 25/25 checks (research contract 6, Fabric sync 2, P2 pair-runner 17). It now verifies payload digest, frozen rate/interval, formal status, queue/latency/achieved TPS, commit counts and drain timestamps. Go benchmark source uses the frozen session duration instead of `planned/rate`; Go build limitation resolved later in the I2 batch: project-local Go 1.25.0 compiled `./cmd/research-fabric-benchmark` successfully. P2 live benchmarking remains uncommissioned. The user has no gateway/sensor connection; none was required or attempted.

R1-specific outcome: `py -3 test/automation/offline_test.py R1` returned `CODE_ONLY_PASS` for 20/20 checks (research contract 6, R1 route/outage parser 14). The parser now binds cloud-route source to the LTE IPv4 and destination to an established MQTT peer; malformed snapshot errors produce retained SHA-256-sealed FAIL evidence. This does NOT exercise gateway outage induction, restoration, LTE route mutation or real connectivity.

A2-specific outcome: `py -3 test/automation/offline_test.py A2` passed 24/24 hardware-free checks (research contract 6, A2 evidence scorer 18). The Fabric-team handoff and exact pre/post-query evidence contract are in

test/automation/authentication/A2-OFFLINE-EVIDENCE-CONTRACT.md. No live endorsement-policy fixture, Fabric transaction or counted A2 sample was created. A2 remains HARNESS_REQUIRED in the operator manual.

I1-specific outcome: `py -3 test/automation/offline_test.py I1` passed 24/24 Python-level checks (contract 6 + I1 scorer 18), including 13/13 direct Node.js VM checks against the exact Node-RED Function-node body. File locations, wiring and evidence requirements are in test/automation/integrity/I1-OFFLINE-QUALIFICATION.md. The function has not been installed in Node-RED and no actual database/Fabric path was tested; I1 stays HARNESS_REQUIRED for live counted trials.

I2-specific outcome: `py -3 test/automation/offline_test.py I2` returned CODE_ONLY_PASS 25/25 (contract 6 + I2 scorer 19). Project-local Go 1.25.0 was installed; the new read-only production-reuse I2 Go helper passed 5/5 direct Go tests and the exact Python-to-Go baseline/tamper/restore cross-language check passed 3/3 phases. The previously uncompiled P2 Go command also compiled successfully. test/automation/integrity/I2-OFFLINE-QUALIFICATION.md preserves command paths and live limitations. No database row, Fabric anchor or sensor was altered.

I3-specific outcome: `py -3 test/automation/offline_test.py I3` returned CODE_ONLY_PASS 39/39 (contract 6 + I3 exact-payload ledger evidence scorer 33). The full synthetic ten baseline + ten same-exact-payload retry + ten conflicting-payload structure passed; negative cases preserve security FAIL when original world state changes, and mark absent independent query/incorrect attack INVALID. test/automation/integrity/I3-OFFLINE-QUALIFICATION.md documents the Fabric team's pre/post-query, identity/anchor, digest, and record-count handoff. Live I3 remains HARNESS_REQUIRED; no ledger transaction was submitted and no synthetic Chapter 4 samples were counted.

Next Step: Audit remaining unqualified operator/manual command surfaces one test at a time; keep published manual statuses unchanged until the full real-world action is commissioned. S2 remains METHODOLOGY_REQUIRED, not solved with synthetic records.

Current checkpoint — 2026-09-21, A1 MQTT code qualification

Current Step: A1 MQTT is the only test under review in this batch. The September 18 isolated broker rehearsal is retained as historical PASS evidence, but the newly hardened code has NOT yet been live-commissioned; do not promote the entire 90-attempt A1 experiment to READY.

Completed: Verified chapter4-results/authentication/mqtt/A1-MQTT-rehearsal-20260918-154556/a1-mqtt-summary.json contains 10 allowed, 10 wrong-password and 10 prohibited-topic trials, zero incorrect decisions, fixture smoke PASS and cleanup PASS. On September 21, ULC-01 answered research-actions-v2|ulc-01 and the isolated authentication listener reported INACTIVE. TCP/22 and later SSH worked after transient handshake timeouts. The administrator SSH key was not usable in noninteractive BatchMode, so the modified server helper and restricted wrapper have not been deployed.

Code corrections in this batch: The trial protocol now emits an explicit observer-delivery field; unauthorized topic delivery is derived from actual observer evidence rather than hard-coded false. Wrong-password trials have their own observer; publication response time ends before the negative observation timeout. The harness validates expected decisions by condition, count and unique trial IDs, latency, observer proof and verified listener post-stop status. Partial-start failures trigger teardown. The server helper refuses to erase runtime material when the isolated service remains active. The repository wrapper/sudoers no longer allow retrieving generated A1 credentials over the restricted research key. Six targeted parser/contract unit tests passed, and both edited shell scripts passed the final `sh -n` check under Ubuntu WSL.

Next Step: Use authorized administrator SSH signing to deploy the updated `mqtt_test_listener.sh`, `server-actions-ssh.sh` and `lorawan-research-actions.sudoers` to ULC-01; validate sudoers and restart only the isolated A1 fixture if needed. Rehearse the exact revised 30-trial MQTT code, verify 10/10 per condition and observed unauthorized delivery 0, check no isolated service/socket or secrets remain, and compare production Mosquitto

pre/post fingerprints. Then update this checkpoint and only the MQTT-subtest qualification status. A1 LoRaWAN and Fabric identity tests remain separate.

Current verified state — 2026-09-18 infrastructure audit

This section supersedes the older P1/OTAA/Fabric blocker notes below. Historical notes remain context, not current readiness.

Current Step

A1 authentication commissioning is in progress. The isolated MQTT third is live-qualified; LoRaWAN is next, followed by the external Fabric identity third.

Completed

- Live readiness at 2026-09-18T03:25:01Z: tooling, counted firmware archive, gateway LTE, clocks, recorder access and full-stack PRE passed.
- ULC-01 is the database leader; ULC-02 and ULC-03 are replicas. All three servers and Gateway-01 were reachable. Collector/verifier readiness endpoints returned HTTP 200; preflight recorded zero evidence_gap and integrity_failure.
- Fresh run: chapter4-results/smoke/manual-P1-rehearsal-20260918-112426. Its exact cloud-UTC measurement window was 03:24:09.233Z–03:25:09.233Z.
- P1 wrapper exited 0: 4 scheduled source attempts, 4 unique ChirpStack accepts (100% PDR in this small rehearsal), 4 calibrated sensor-to-database samples, and 4/4 Fabric confirmations with transaction IDs.
- Observed uplink frequencies: 923.2 and 923.4 MHz. This is observed traffic, not a complete regional-configuration audit.
- Fabric settlement completed in 215.695 seconds within the 420-second allowance, after the measurement window closed. Do not shorten the observation window or count settlement traffic as new source attempts.
- All 62 evidence-manifest entries verified. Summary regeneration twice in a temporary copy reproduced every JSON field except relocated run_dir and byte-identical CSV. Original evidence remained sealed.
- The verifier rejected an intentionally modified CSV in a temporary fixture. Recorder state and ownership lock were clean after completion.
- Fixed ensure_ready.py so --skip-live-pre reports technical_gate=SKIPPED on successful tooling-only checks, rather than implying live PASS. Four targeted cases passed on the host: live success, live failure, tooling-only success and tooling failure.
- Added research-recorder/verify-run-reproducibility.py. Instructions are in this directory's README.md; machine-readable evidence is REPRODUCIBILITY-20260918.json.
- A1 MQTT commissioning passed on ULC-01 using the restricted research-actions-v2 fixture. Rehearsal chapter4-results/authentication/mqtt/A1-MQTT-rehearsal-20260918-154556 recorded exactly 30 trials: 10 allowed, 10 wrong-password, and 10 prohibited-topic decisions. All 30 matched expectation, false acceptance count was zero, smoke and cleanup passed, and the temporary listener was confirmed inactive afterward.
- Production Mosquitto isolation was verified with byte-identical pre/post process/config/listener fingerprints (SHA-256 ad2833f211f9e7db674c54df45b4b42dd25d6212987ddf60897a7357f579a4a3); port 1884 was absent after teardown. This commissions only the MQTT third of A1, not the whole A1 experiment.

Next Step

Proceed with formal P1 repetitions or commission the remaining test harnesses individually. Only PRE and P1 are released READY commands. No counted 30-minute P1 repetition was performed by this audit. S2 still requires a specified methodology; other unreleased test IDs retain their current harness/capture restrictions.

Operational notes

Use a persistent managed process for the interactive P1 command so capture and finalization survive individual MCP request timeouts. The 60-second rehearsal is selected explicitly with `RESEARCH_MANUAL_REHEARSAL_SECONDS=60` and is non-counted. Allow the process to finish; a silent interval can include preflight, warm-up, settlement and evidence export.

During this audit, lorawan-setup-scoped MCP calls stalled while commands through the same jervis-hijo host's authorized rel-ai-mcp workspace returned successfully using the absolute LoRaWAN project path. A transport timeout alone did not establish an infrastructure outage. Check existing process state and saved results before retrying an experiment.

The workstation was approximately 91 seconds ahead of cloud UTC. The recorder independently calibrated the source clock at both boundaries; the measured drift was about 1.24 ms. This is calibrated measurement support, not a claim that the workstation clock is synchronized.

Reproducibility here means deterministic processing of these recorded inputs with the recorded Python version and summarizer hash. It does not prove fresh-machine deployment, repeatable RF outcomes, full HA failover, or readiness of every Chapter 3 experiment.

This file is the handoff point for continuing the Chapter 3 research-test manual work in a new chat.

Current rule

- `chapters/Zacarias_Chapter3.pdf` controls the required experiments, counts, durations, workloads, measurements, formulas, and stated acceptance/security criteria.
- Verified live infrastructure controls how those experiments are executed.
- Stale architecture/topology prose from Chapter 3 is not implementation authority.
- The operator manual must stay simple.
- A test code block is included in the operator manual only when the complete action is currently runnable through the commissioned harness.
- Syntax-only checks, guard-only commands, intentional `BLOCKED` snippets, and missing-fixture placeholders do not qualify as working operator blocks.
- Tests that are still incomplete remain documented with their full Chapter 3 measurement requirements, but no operator code block is published until the harness passes.

Verified platform state before this manual audit

On 2026-09-16 the live technical readiness gate passed with the counted EMU-01 profile, Gateway-01 LTE production path, ULC cloud stack, database leader check, evidence services, clock gate, and restricted recorder access healthy. EMU-01 was present on COM11 and the security node on COM16. The recorder reported zero `evidence_gap` and zero `integrity_failure` statuses during the fresh preflight.

Chapter 3 coverage audit completed

The manual covers P1, P2, R1, R2, A1, A2, S1, S2, F1, F2, I1, I2, I3, T1, and T2. Important methodology notes retained:

- P2 requires four Fabric workloads (1 tx/15 s, 1 TPS, 5 TPS, 10 TPS), 5 minutes each, three repetitions. Chapter 3 requires a with-vs-without-Fabric comparison but does not prescribe the no-Fabric comparison count. Any chosen comparison design must be frozen before counted execution.

- S2 application-layer duplicate/replay is listed in Chapter 3 but lacks a standalone count, timing, injection boundary, metric formula, and pass rule. It remains methodology-blocked and must not be invented or substituted with I3.
- Resource measurements follow the current participating machines while preserving the Chapter 3 CPU/memory/bandwidth metrics.

Operator-block audit in progress

The previous validation logic reported PASS even for tests whose status was HARNESS_REQUIRED, CAPTURE_READY, or METHODOLOGY_REQUIRED. That is being corrected. The new publication rule is stricter: only complete READY tests may expose a code block in the final manual.

Current formal readiness before this cleanup:

- PRE: READY
- P1: READY
- P2: HARNESS_REQUIRED
- R1: CAPTURE_READY
- R2: HARNESS_REQUIRED
- A1: CAPTURE_READY
- A2: HARNESS_REQUIRED
- S1: CAPTURE_READY
- S2: METHODOLOGY_REQUIRED
- F1: CAPTURE_READY
- F2: CAPTURE_READY
- I1: HARNESS_REQUIRED
- I2: HARNESS_REQUIRED
- I3: HARNESS_REQUIRED
- T1: CAPTURE_READY
- T2: CAPTURE_READY

Current step

1. Change readiness validation so non-ready experiments are never shown as code-block PASS.
2. Rehearse the released PRE/P1 entry points against the live environment.
3. Replace the long manual with a simple measurement-first format.
4. Keep code blocks only for released READY tests.
5. Regenerate and render-check the PDF.

Next engineering work after this cleanup

Finish and commission missing one-block harnesses one experiment at a time. When a harness becomes complete, run its non-destructive interface checks plus a safe rehearsal/canary, update readiness to READY, then add exactly one simple operator block to the manual. Do not publish a block first and hope the fixture works later.

2026-09-16 operator-block verification update

- The manual was reduced to a simple measurement-first format. Non-ready tests retain their Chapter 3 design and measurements but have no placeholder operator commands.
- `ensure_ready.py` now validates only operator blocks actually released in the manual instead of reporting guard-only non-ready tests as code-block PASS.
- The exact PRE manual command was executed live and passed: technical gate, tooling compile, MQTT self-test, counted firmware archive, recorder state, Gateway-01 LTE invariant, and live full-stack PRE were all PASS.
- The first 15-second P1 rehearsal proved recorder lifecycle/finalization and produced sealed evidence, but it contained zero EMU-01 `SENSOR_TX` attempts. Formal PDR was correctly `WITHHELD_NO_SENSOR_TRANSMISSION_ATTEMPTS`; therefore that rehearsal does not prove P1 measurement readiness.
- An ad-hoc 60-second serial audit temporarily created its own COM11 lock because its launcher left a child `serial_capture.py` process alive after the parent was killed. Both audit processes were terminated. This was an audit-side process-lifecycle artifact, not accepted as a production recorder defect.
- `run_test.py` has now been hardened so a P1 rehearsal/formal run cannot return success merely because capture stopped cleanly: it must have source attempts and `MEASURED_CHIRPSTACK_ACCEPTANCE_BOUNDARY`. A formal 30-minute P1 additionally requires at least 100 source attempts before it can return success.
- A clean 60-second P1 rehearsal is now the publication gate. If it does not produce real EMU-01 source attempts plus measurable ChirpStack PDR, P1 must be removed from the released manual blocks until the sensor path is repaired.

Final code-block publication result for this audit

- PRE remains released. Its exact manual command was executed live and passed the full technical gate, including Gateway-01 LTE and live PRE.
- P1 is no longer released. A clean 60-second rehearsal (`manual-P1-rehearsal-20260916-160437`) ran through the real recorder/finalization path with healthy supervisor checks, but EMU-01 produced 0 `SENSOR_TX` attempts and Gateway-01 concentrator reported `rx_received: 0 / rx_received_ok: 0`. The hardened P1 entry point therefore exited nonzero with P1 measurement boundary invalid: `source attempts=0, need >= 1`.
- `run_test.py` now fails P1 when required source/PDR evidence is missing; a formal P1 would also require at least 100 source attempts before returning success.
- Because the user requires every manual code block to be working, the P1 code block was removed from the manual and P1 status changed to `CAPTURE_READY`; live sensor transmission boundary not proven.
- The final manual now contains exactly one operator code block: PRE. All other tests remain fully documented for measurements/design but have no command until their complete harness and required observation boundary pass.

Next technical task if continuing beyond manual cleanup: diagnose the current EMU-01 OTAA/RF state, restore real 15-second `SENSOR_TX` + gateway reception, rerun a clean P1 rehearsal, and only then re-release P1.

Final manual state after regeneration

- Full live `ensure_ready.py` completed PASS at 2026-09-16T08:09:56Z: technical gate, tooling compile, MQTT self-test, counted firmware archive, recorder state, Gateway-01 LTE invariant, and live PRE all PASS.
- Published operator blocks: PRE only. `CURRENT-READINESS.md` lists PRE = READY / PASS and no other released block.

- Final Markdown audit: exactly 1 PowerShell code block (2 fence lines), PRE command count = 1, P1 command count = 0, P1 withheld-status marker present.
- Simplified PDF regenerated from the final Markdown. SHA-256:
404281D2641EBE4E86470582653A11F32C1130ECEAA6184AD75B1D4D798D0F3F; size 258,966 bytes.
- Browser layout inspection of the rendered manual confirmed a readable simple layout, intact status table, PRE command visible, and P1 marked not released.

Resume rule: never add a test command to the manual merely because its interface or guard validates. Add it only after the complete test action and required observation boundary pass a real rehearsal/canary. The next unreleased candidate is P1, but first repair/prove EMU-01 OTAA/15-second RF traffic.

2026-09-16 per-test code-block audit

- Exact published PRE block executed live and PASS.
- All 16 test IDs passed their safe wrapper/interface validation path.
- Manual still contains exactly one released PowerShell block: PRE.
- P1 remains withheld because live measurement boundary is not proven.
- Detailed audit: test/automation/research-manual/CODE-BLOCK-AUDIT-20260916.md

2026-09-16 commissioning continuation

- A durable commissioning task was opened to make every Chapter 3 test a complete fail-closed one-command action before publication.
- EMU-01 COM11 was initially blocked by orphaned `serial_capture.py` child processes left after stopping a diagnostic parent process. Both child processes were terminated and an exclusive COM11 open then passed. This process-tree cleanup gap must be hardened in the recorder tooling.
- The workstation Arduino/RAK toolchain is installed even though `arduino-cli` is not on PATH. Verified CLI: Arduino CLI 1.5.1; installed RAKwireless nRF52 core 1.3.3; FQBN `rakwireless:nrf52:WisCoreRAK4631Board`.
- The archived counted EMU-01 15-second image was hash-verified against its build record before upload. The image itself was not rebuilt or altered.
- The first Arduino upload attempt failed because the orphan serial process still owned COM11. A later PowerShell wrapper incorrectly treated normal DFU stderr as a terminating PowerShell error and left the node in bootloader mode on COM12. Direct invocation of the RAK/Adafruit DFU executable then completed successfully with exit code 0 and Device programmed.
- A bounded 75-second post-flash serial proof on the application port then returned `EMU01_OTAA_JOIN=FAIL, SENSOR_TX_COUNT=0`. This proves the counted firmware boots and executes but the LoRaWAN join currently fails. The active P1 blocker is therefore the OTAA/RF/network-server join boundary, not firmware programming, USB enumeration, recorder startup, or LTE backhaul.
- Gateway-01 production backhaul remained healthy during this work: LTE `wwan0` is up and registered, default route metric 10 is active, the cloud route uses `wwan0`, and MQTT mTLS to port 8883 is established.
- Historical verified configuration names EMU-01 DevEUI `ac1f09ffffe296d29` and JoinEUI `0000000000000000`; the current protected AppKey must be compared to the live ChirpStack root key without writing either secret into repository documentation or chat output.
- Next P1 action: compare protected EMU-01 DevEUI/JoinEUI/AppKey and AS923 settings against the live ChirpStack device/profile/root-key state and inspect the latest gateway/ChirpStack join evidence. Fix the first mismatched side, then require OTAA PASS + at least two 15-second `SENSOR_TX` records + gateway/ChirpStack reception before rerunning the full P1 recorder rehearsal.

2026-09-17 P1 live recovery and Fabric blocker

- EMU-01 is now joined and transmitting the counted profile correctly. Bounded COM11 capture observed consecutive 15-second `SENSOR_TX` records with `join=1` and `send_status=0`.
- Gateway-01 simultaneously receives the matching AS923 RF cadence. After joining, ADR converged toward SF7 while LTE `wwan0` remained the production cloud path and MQTT mTLS stayed established.
- `run_test.py` gained an explicit non-counted P1 commissioning mode: setting `RESEARCH_MANUAL_REHEARSAL_SECONDS` intentionally permits an unreleased P1 rehearsal in the `smoke` group without weakening the normal guard or marking P1 READY.
- Rehearsal `manual-P1-rehearsal-20260917-083631` is invalid for publication because the outer command timeout killed the wrapper before orderly stop. The source window contained four valid attempts but remote evidence continued until recovery, producing a mismatched observation window. Recorder recovery subsequently cleared all children and the active marker. Treat this run as diagnostic only.
- Clean rehearsal `manual-P1-rehearsal-20260917-084445` completed normally with recorder CLEAN. It measured 4 source attempts, 4 ChirpStack accepts, formal PDR 100%, exact payload/FCnt correlation, populated gateway/server resource samples, and complete telemetry measurements. The P1 radio/application/measurement boundary is therefore proven.
- P1 is still NOT released because the Fabric portion of the Chapter 3 contract failed live. All four rehearsal outbox rows remained pending with no Fabric TxID/commit/block data after the run. A later read-only query still showed those same rows pending; global evidence status was `pending=15`, `verified=103`, `evidence_gap=0`, `integrity_failure=0`, and `LATEST_VERIFIED_AT=2026-09-17 00:45:45.470188+00`, before the first P1 event.
- Both Fabric-adapter containers report Up, but run-window adapter logs are empty and the queue is not advancing. Container-up state is therefore insufficient; the active blocker is the Fabric adapter `worker/claim/credential/lease` path.
- Do not promote P1 or publish its manual command until the Fabric queue resumes, the exact P1 events reach a measurable terminal Fabric state, and a new clean P1 rehearsal proves Fabric transaction success in addition to PDR/resources.
- Next action: diagnose adapter worker state, OpenBao/AppRole rotation, lease/claim state, and Fabric Gateway reachability using existing authorized control paths; repair minimally; verify pending decreases and verified advances; then harden the P1 publication validator to require Fabric completion before rerunning the clean rehearsal.

2026-09-17 Fabric outbox eligibility diagnosis

- Live ULC-01 monitoring proves the Fabric adapter is enabled and HTTP-ready, not dead: adapter readiness returned HTTP 200 while the database contained 292 outbox rows (`confirmed=69`, `pending=222`, `processing=1`). Collector readiness was also HTTP 200.
- A 24-hour EMU-01 outbox export found 195 telemetry-attestation-v2 rows. Every one was still pending, every one had `attempts=0`, and every `last_error_category` was empty. New rows continued arriving on the expected ~15-second cadence. This proves the adapter is not reaching Fabric and being rejected; these rows are not being claimed at all.
- A matching recent gateway packet-flow export found 199 EMU-01 rows: 185 already had evidence status verified, 14 were still pending with reason `journal_source_missing`, and all 199 outbox rows were pending. Therefore evidence verification lag explains only the newest minority and cannot explain the main Fabric backlog.
- The current adapter claim predicate was inspected in `evidence-services/cloud/internal/fabricadapter/repository.go`. A normal row is claimable only when status is `pending/failed` and `next_attempt_at <= now()`, `finalized_payload` IS NOT NULL, and (for telemetry-attestation-v2) a matching `gateway_evidence.event_verification` row is verified. Since 185 recent EMU

rows are verified but remain attempts=0, the immediate fault boundary is a remaining eligibility condition, most likely missing `finalized_payload` or scheduling state, not OpenBao/Fabric transaction rejection.

- Do not chase Fabric Gateway or OpenBao errors until the unclaimed-row eligibility failure is proven. Next action: trace the production outbox insert/finalization path, prove which claim prerequisite is absent on live EMU rows, repair that upstream durable write/state transition, drain the backlog, then rerun P1 only after new rows are claimed and committed.

Supporting runbook: ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Zacarias Chapter 3 — Authoritative Research Test and Metrics Source of Truth

Authority and scope

This document is the testing authority derived from `chapters/Zacarias_Chapter3.pdf` (74 PDF pages).

Use the new chapter as authoritative for the research questions, experimental conditions, trial/run counts, durations, metrics, evidence requirements, analysis rules, and stated success/security criteria only.

Do **not** use its architecture diagrams, host placement, service placement, topology, deployment descriptions, component ownership, paths, addresses, or other implementation prose as current infrastructure truth. Those descriptions may be stale. The commissioned system must be tested as it actually exists, using verified live configuration and current deployment documentation, while preserving the experimental condition and measurement intent defined here.

When sources disagree:

1. This document controls **what formal research test is required and what must be measured**.
2. The new `Zacarias_Chapter3.pdf` is the provenance source for those requirements.
3. Verified live state and current deployment/runbooks control **how the commissioned system implements the test**.
4. Older Chapter 3/4/5 drafts and obsolete architecture prose do not override either of the above.

A test must not be silently changed because the current architecture differs from the dissertation prose. Adapt the injection/observation method to the live system while keeping the intended independent variable, observation boundary, metric, and evidence intact.

Formal experiment inventory

ID	Formal experiment	Required design from Zacarias Chapter 3	Primary chapter pages
P1	End-to-end normal-operation performance	3 runs × 30 min; legitimate sensor every 15 s; ≈120 transmissions/run; no attacks/outages/failures	PDF pp. 17–21

ID	Formal experiment	Required design from Zacarias Chapter 3	Primary chapter pages
P2	Hyperledger Fabric performance and overhead	4 workloads: 1 tx/15 s, 1 TPS, 5 TPS, 10 TPS; 5 min/condition; 3 repetitions = 12 workload runs; compare processing with vs without Fabric	PDF pp. 21–27
R1	Internet-connectivity interruption and recovery	3 × 2-hour runs: 30 min normal + 60 min external-Internet interruption + 30 min recovery; sensor remains at 15 s cadence	PDF pp. 31–34
R2	Database–blockchain consistency and Fabric recovery	10 normal records; 10 additional records while Fabric unavailable; reconcile the same 10 pending records after restoration	PDF pp. 35–41
A1	Authentication and access control	9 conditions × 10 trials = 90 attempts: 3 LoRaWAN + 3 MQTT + 3 Fabric identity/authorization conditions	PDF pp. 43–46
A2	Fabric endorsement-policy enforcement	10 normal valid transactions + 10 policy-violation attempts + 10 post-restoration valid transactions = 30 attempts	PDF pp. 46–50
S1	LoRaWAN replay and spoofing	Replay: 10 legitimate controls + 10 gateway-received replay attempts. Spoofing: 10 genuine controls + 10 gateway-received forged/invalid-MIC attempts. Total = 40 counted attempts	PDF pp. 50–55
S2	Application-layer duplicate/replay	Listed in Table 3.4 as a proposed test, but the chapter provides no standalone detailed procedure, trial count, duration, metric formula, or acceptance rule for it. Treat as an unresolved methodology requirement; do not silently omit it and do not invent counted results.	PDF pp. 28–30
F1	MQTT connection flooding	normal 0/s, moderate 10 invalid connections/s, high 50/s; 5 min/condition × 3 repetitions; 5-min recovery after every run	PDF pp. 55–61
F2	Invalid application-message flooding	normal 0/s, moderate 10 invalid messages/s, high 50/s; 5 min/condition × 3 repetitions; 5-min recovery after every run	PDF pp. 55–61
I1	Application-layer data alteration	10 unchanged controls + 10 deliberately altered records	PDF pp. 61–65

ID	Formal experiment	Required design from Zacarias Chapter 3	Primary chapter pages
I2	Post-storage database tampering	10 unchanged controls + 10 deliberately modified stored records	PDF pp. 62–65
I3	Blockchain duplicate / unauthorized overwrite	Establish 10 valid baseline anchors; submit the same 10 trace IDs with same hashes (duplicate attempts); submit the same 10 trace IDs with different hashes (overwrite attempts)	PDF pp. 66–68
T1	Individual sensor-record traceability	10 individual-record trace trials	PDF pp. 69–72
T2	Device-history reconstruction	10 sequences × 5 consecutive readings = 50 records	PDF pp. 69–73

Traceability total: **20 trials covering 60 records**. Flooding total: **18 five-minute load runs** across two flood types, with a five-minute recovery observation after each run.

Chapter-internal consistency notes

The new chapter contains a few internal inconsistencies that must be handled explicitly instead of guessed:

- **Application-layer duplicate/replay:** Table 3.4 lists this as a proposed replay/spoofing test and the surrounding prose says replay/spoofing assessment includes duplicate records, but the detailed replay section and Table 3.8 define only LoRaWAN replay and invalid-MIC spoofing. Because no count, duration, exact injection method, metric formula, or success threshold is supplied for the application-layer duplicate/replay item, it remains a **methodology-resolution blocker** for that specific test. It must either be formally specified by the research team/adviser or explicitly removed from the methodology before final completion. Existing duplicate detection in I3 or database deduplication must not be silently reported as this missing test unless the methodology is formally amended.
- **Integrity-count wording:** the introduction to Section 3.2.4.2.3 says the integrity testing yields “40 total test attempts,” which matches the detailed I1/I2 design (10 unchanged + 10 altered at application layer, plus 10 unchanged + 10 tampered post-storage). The later I3 blockchain-layer procedure is an additional design using 10 already-valid baseline anchors followed by 10 duplicate and 10 conflicting-overwrite submissions. Therefore report I1/I2 as 40 counted application/post-storage attempts and report I3 separately; do not collapse them into an unsupported single “40 total including I3,” and do not invent a grand-total count without stating whether the 10 I3 baseline anchors are fixtures or counted controls.
- **R1 blockchain behavior:** Chapter 3 assumes blockchain verification can continue locally during external-Internet loss because of its older deployment model. In the commissioned system Fabric is an external dependency. Preserve the **R1 independent variable and metrics** (external-Internet interruption, local monitoring continuity, loss/duplication/order/latency/recovery) but report actual Fabric state truthfully; queued/PENDING work is not a Fabric commit.
- **P2 no-Fabric comparison count:** the chapter fixes the Fabric workload runs at four rates × three five-minute repetitions, but it does not state how many matching “without Fabric” comparison runs are required for the overhead equations. Before P2 counted testing, freeze and document the comparison design (which workload(s), duration, repetitions, warm-up/stabilization, and pairing method) so additional-latency/CPU/memory overhead cannot be selected post hoc.
- **Synthetic-versus-physical test data:** the chapter explicitly calls for synthetic/known test readings in authentication, LoRaWAN replay, integrity, and traceability procedures, while the current project sensor manuals deliberately use real physical Agriculture Kit readings. This is a methodology divergence, not an architecture detail. Before those counted tests, formally choose and document the data provenance for each

experiment. Do not label real physical readings “synthetic,” and do not replace a required known-value/synthetic fixture with uncontrolled physical values unless the methodology is explicitly amended while preserving the same security condition and identifiers.

Measurement rules and units

Use raw timestamps/counters in their native precision and preserve them in evidence. Report human-facing metrics with explicit units; never show an unlabeled number.

Recommended reporting normalization, without changing the chapter definition:

- latency, response, verification, reconciliation, retrieval, and recovery time: retain raw timestamps; calculate in seconds and report milliseconds where useful;
- CPU utilization: percent of available processing capacity, with average and maximum;
- memory: used bytes and total bytes plus utilization percent; dashboards may show MiB/GiB used / total;
- network bandwidth: retain RX/TX byte counters and derive rate in Mbps over the measurement interval;
- throughput: records/s for system processing and transactions/s (TPS) for Fabric; records/min may additionally be shown for the slow 15-second sensor baseline;
- transaction/block size: bytes, with KiB only as a display conversion;
- rates: percent with numerator and denominator retained;
- counts: integer counts, never converted to percentages without preserving the denominator.

P1 — End-to-end normal-operation metrics

Required measures:

- **Packet-delivery rate (PDR)** = $N_{\text{received_by_ChirpStack}} / N_{\text{transmitted_by_sensor}} \times 100$.
- **End-to-end latency per reading** = $T_{\text{database_storage}} - T_{\text{sensor_transmission}}$.
- **Mean end-to-end latency** across successfully delivered readings.
- **System throughput** = $N_{\text{readings_successfully_processed_and_stored}} / \text{test_duration}$.
- **Fabric transaction success rate (TSR)** = $N_{\text{valid_committed}} / N_{\text{submitted}} \times 100$.
- **Average CPU utilization** and **maximum CPU utilization**.
- **Memory utilization per observation** = $\text{memory_used} / \text{memory_total} \times 100$.
- **Average memory utilization** and **maximum memory utilization**.

Evidence must permit correlation among the sensor transmission, ChirpStack acceptance, database row, and Fabric result. A transaction ID alone is not proof of a successful commit.

Timing caveat: sensor→database latency is valid only when the source timestamp and database clock are demonstrably correlated. If that timing boundary cannot be proven for a run, do not invent the Chapter 3 metric; classify the timing evidence as unavailable/invalid and repair the clock/correlation gate before formal measurement.

P2 — Hyperledger Fabric performance and overhead metrics

For each workload (1/15 s, 1 TPS, 5 TPS, 10 TPS), collect:

- **Commit latency per transaction** = $T_{\text{commit}} - T_{\text{submission}}$.
- **Mean commit latency**.
- **Transaction throughput** = $N_{\text{committed}} / \text{duration_seconds}$ TPS.
- **Transaction success rate** = $N_{\text{committed}} / N_{\text{submitted}} \times 100$.

- **Average transaction size** in bytes.
- **Average block size** in bytes.
- **Average / maximum CPU utilization**.
- **Average / maximum memory utilization**.

For the same processing workload with and without Fabric submission, calculate. **The chapter does not specify the count/repetition structure of the no-Fabric comparison, so that comparison design must be frozen before counted P2 execution and retained in run metadata:**

- **Additional latency** = $L_{\text{with_Fabric}} - L_{\text{without_Fabric}}$.
- **Percentage latency increase** = $(L_{\text{with_Fabric}} - L_{\text{without_Fabric}}) / L_{\text{without_Fabric}} \times 100$.
- **CPU overhead** = $\text{CPU}_{\text{with_Fabric}} - \text{CPU}_{\text{without_Fabric}}$.
- **Memory overhead** = $\text{Memory}_{\text{with_Fabric}} - \text{Memory}_{\text{without_Fabric}}$.

Only valid transactions are used for the performance load. Invalid/rejection traffic belongs to the security tests, not P2.

Resilience tests

R1 — External-Internet interruption and recovery

This is a formal test in the new chapter and is **not equivalent to Fabric-only endpoint isolation**.

Each run:

- 30 min normal operation ≈ 120 readings;
- 60 min external-Internet interruption ≈ 240 readings;
- 30 min recovery ≈ 120 readings;
- repeat the complete two-hour run 3 times.

The intended fault is loss of the system's **external Internet connection while local monitoring functions remain powered and operational**. The exact live implementation must isolate the WAN dependency without accidentally injecting an unrelated failure.

Required measures, separately for normal/interruption/recovery periods:

- local service availability;
- data-delivery rate = $\text{transmitted readings successfully received/stored} \div \text{transmitted readings} \times 100$;
- data-loss count;
- duplicate-record count;
- database–blockchain consistency rate;
- end-to-end latency;
- recovery time from restoration until external connectivity returns to normal;
- chronological accuracy using timestamps/frame counters.

Use frequencies/percentages for availability, delivery, missing/duplicate records, chronology, and database–blockchain consistency. Report mean and standard deviation for latency and recovery time.

Research success criterion: essential local monitoring continues through the interruption and normal connectivity returns without database–blockchain inconsistency. Compare loss/duplication/linkage behavior against the normal baseline so ordinary LoRaWAN loss is not falsely attributed to the outage.

R2 — Fabric unavailability, database–blockchain consistency, and reconciliation

This is a **separate** resilience test from R1.

Stages:

1. Normal control: 10 valid readings should move PENDING → VERIFIED only after confirmed Fabric commitment.
2. Fabric unavailable: 10 additional valid readings remain locally stored and **PENDING**; no fabricated Fabric transaction ID and no false VERIFIED state.
3. Fabric restored: reconcile those same 10 pending records; do not create new recovery test records.

For every record retain at least trace ID, database-storage result, initial/final blockchain state, submission result, Fabric transaction ID when applicable, original hash, recomputed database hash, blockchain hash, and timestamps.

Metrics:

- pending-record identification rate = correctly PENDING outage records ÷ outage records × 100;
- recovery success rate = pending records successfully reconciled/VERIFIED ÷ pending records requiring reconciliation × 100;
- post-recovery hash-consistency rate = recovered records with matching DB/Fabric hashes ÷ recovered records verified × 100;
- false-verification count;
- orphaned database record count;
- missing blockchain record count;
- duplicate blockchain record count;
- conflicting-hash count;
- failed-reconciliation count;
- records remaining PENDING after recovery;
- recovery time from Fabric restoration until every recoverable pending record is VERIFIED or classified FAILED;
- optionally mean per-record reconciliation time.

Required criteria stated by the chapter:

- 100% pending-record identification;
- zero false verifications;
- 100% recovery success for valid pending records;
- 100% post-recovery hash consistency;
- zero missing or duplicate valid blockchain integrity anchors.

Authentication and authorization

A1 — 90 authentication/access-control attempts

LoRaWAN, 10 trials each:

1. registered device + correct OTAA credentials → allow/join/uplink;
2. registered DevEUI + incorrect AppKey → reject;
3. unregistered DevEUI → reject/not activated.

MQTT, 10 trials each:

1. authorized account + allowed topic → accept;
2. incorrect password → reject connection;
3. valid limited account + prohibited topic → authenticate as applicable but deny publish.

Fabric identity/authorization, 10 trials each:

1. authorized writer → endorse/commit;
2. valid identity without writer permission → reject, unchanged world state;
3. invalid/untrusted identity → reject, unchanged world state.

Metrics:

- authorized success rate;
- unauthorized rejection rate;
- false-acceptance count;
- false-rejection count;
- correct-decision rate;
- response time per layer;
- unauthorized state-change count.

Response boundaries:

- LoRaWAN: JoinRequest → JoinAccept or final rejection decision;
- MQTT: connection/publish request → broker response;
- Fabric: transaction submission → commitment or rejection.

Use frequencies/percentages for decisions and mean + standard deviation for response time.

Required security criterion: **zero false acceptance and zero unauthorized state change**.

A2 — Fabric endorsement-policy enforcement

Three conditions, 10 attempts each:

1. required endorsers available; valid transactions should commit;
2. required endorsement missing; policy-violating transaction must not create valid world-state change;
3. required endorser restored; valid transactions should again commit.

Metrics:

- endorsement-policy enforcement rate;
- valid transaction success rate;
- post-restoration recovery transaction success rate;
- unauthorized state-change count.

Required criterion: **100% policy enforcement and zero unauthorized state changes** during the violation condition; valid transactions should resume when required endorsement is available.

Replay and spoofing

S1 — LoRaWAN replay and spoofing

A counted attack attempt is valid only after the gateway proves RF reception. A transmit command by the attack node is not sufficient.

Replay: 10 legitimate packets + 10 corresponding received replays using an old frame counter. The legitimate device must advance its counter before the old captured frame is replayed.

Spoofing: 10 genuine packets + 10 received forged packets using the legitimate address identity but an invalid MIC/no legitimate session key.

Required measures:

- replay-rejection rate;
- spoofing-rejection rate;
- false-acceptance count;
- false-rejection count;
- unauthorized-propagation rate into MQTT/Node-RED/database/Fabric;
- processing time from gateway reception to ChirpStack accept/reject when timestamps permit.

Required criterion: **zero false acceptance and zero unauthorized propagation**; all legitimate controls should be processed.

S2 — Application-layer duplicate/replay methodology gap

Table 3.4 separately proposes resubmitting a previously processed or duplicate sensor record at the application layer. The detailed methodology never specifies its trial count, repetition count, timing, exact duplicate identifier, expected downstream decision, or analysis formula. Until that ambiguity is formally resolved, classify S2 as **BLOCKED_BY_METHODODOLOGY** rather than PASS/FAIL/INVALID. Preserve any available duplicate-prevention evidence, but do not promote opportunistic deduplication observations into counted S2 research data.

At minimum, a later formal specification must define before execution: the canonical record/identifier to duplicate, how many control and duplicate trials are required, which layer must demonstrably receive the duplicate, whether rejection or idempotent recognition is expected, which database/Fabric state must remain unchanged, the timing boundary if a decision-time metric is required, and the numerator/denominator for any duplicate-rejection or unauthorized-propagation rate.

Flooding / DoS

Run two independent flood types:

1. MQTT invalid-connection flooding;
2. invalid application-message flooding.

For each: normal 0/s, moderate 10/s, high 50/s; 5 minutes each; 3 repetitions; 5-minute recovery observation after every run. Legitimate sensor traffic continues every 15 seconds (≈ 20 legitimate readings per five-minute window).

A short pilot may adjust a rate only if the proposed rate causes immediate failure or no measurable resource effect. **After the pilot, freeze the selected moderate/high rates for all repetitions.**

Required measures for every run:

- invalid-traffic rejection rate;
- legitimate-message delivery rate;
- legitimate end-to-end latency;
- valid-message throughput;
- average and maximum CPU utilization;
- average and maximum memory utilization;
- network RX/TX bandwidth;
- service availability;
- unauthorized database/Fabric record count;
- recovery time from flood stop until latency/resource behavior returns to the predefined normal range.

Report mean + standard deviation for latency, throughput, CPU, memory, bandwidth, and recovery time; frequencies/percentages for delivery, rejection, availability, and unauthorized records.

Required security behavior: invalid traffic is rejected, no invalid message becomes a valid DB/Fabric record, legitimate traffic continues, and there is no sustained service failure after recovery. A service crash, unauthorized record, or sustained legitimate-processing failure is a flooding-related security failure.

Data integrity

I1 — Application-layer alteration

10 unchanged controls + 10 deliberately altered records after the initial hash but before valid storage.

Expected: unchanged hashes match and pass; altered hashes mismatch and the altered record is blocked/quarantined, not stored/submitted as valid.

I2 — Post-storage database tampering

10 unchanged stored controls + 10 controlled post-storage value changes after the original hash has been committed.

Expected: unchanged DB hash matches Fabric; each modified DB record recomputes to a different hash and triggers integrity detection.

I1/I2 metrics:

- integrity-verification rate;
- tamper-detection rate;
- false-positive count;
- false-negative count;
- unauthorized-storage count;
- verification time.

Use frequencies/percentages for decisions and mean + standard deviation for verification time.

Required criterion: **zero false negatives and zero unauthorized storage**; all unchanged records match, all deliberately altered records mismatch.

I3 — Blockchain duplicate / overwrite integrity

Create 10 valid, confirmed baseline integrity anchors with unique `trace_id` values. Then:

- duplicate condition: resubmit each of the 10 using the same `trace_id` and same hash;
- overwrite condition: resubmit each of the same 10 `trace_id` values with a deliberately different hash using an otherwise authorized identity.

After every attempt, query world state and prove the original hash remains unchanged.

Metrics:

- duplicate-rejection rate;
- overwrite-rejection rate;
- original-hash preservation rate;
- unauthorized state-change count.

Expected behavior: duplicate creates are rejected, conflicting overwrites are rejected, and each original integrity anchor remains unchanged.

Traceability

T1 — Individual record trace

10 known readings. Each trial must reconstruct the reading from its original LoRaWAN/application evidence through the database record and corresponding Fabric evidence.

T2 — Device-history reconstruction

10 non-overlapping sequences of 5 consecutive readings each. Reconstruct all five in the correct order and link each to its blockchain hash/transaction.

Required trace fields include at least:

- trace identifier;
- Device EUI;
- LoRaWAN frame counter;
- sensor type/value/unit;
- event timestamp;
- gateway identifier;
- database record identifier;
- SHA-256 hash;
- Fabric transaction identifier;
- Fabric commit timestamp or block reference.

Metrics:

- trace-retrieval success rate;
- record-completeness rate;
- database–blockchain linkage rate;
- chronological-order accuracy;
- missing-record count;
- duplicate-record count;
- individual trace-retrieval time;
- device-history reconstruction time.

Use frequencies/percentages for retrieval/completeness/linkage/order and counts; mean + standard deviation for retrieval/reconstruction time.

Required criterion: complete retrieval and correct linkage of all tested records; no expected record missing, duplicated, or placed in the wrong order.

Universal evidence and classification rules

For every formal run/trial retain enough raw evidence to answer:

1. What exact experimental condition was intentionally created?
2. Was the condition proven to have reached the component/layer being tested?
3. What result was expected before the test?
4. What actually happened?
5. Which raw log/CSV/database/Fabric record proves it?
6. Which numerator and denominator produced every reported rate?
7. Which timestamps define every reported latency/recovery interval?

Classification:

- **PASS** — valid condition, expected behavior, complete evidence;
- **FAIL** — valid condition, unexpected behavior; retain it as research data;
- **INVALID** — intended condition or required evidence was not achieved; retain and rerun under a new ID;
- **BLOCKED** — an explicitly required external fixture/authorization is unavailable before the test begins.

Never delete failed/invalid attempts to improve results. Screenshots are supporting evidence only; raw logs, counters, exports, hashes, and transaction evidence are authoritative.

Audit of current project manuals against the new chapter

The existing test track already covers much of the new methodology, but the following mismatches must be corrected before the affected counted test is executed:

Area	Current project state	Required action
P1 baseline	Recorder boundary corrected in source	summarize-run.py now derives formal PDR from unique accepted target-device ChirpStack deduplication_ids across both HA ChirpStack logs divided by captured scheduled SENSOR_TX attempts; a scheduled source-side send failure remains in the denominator. Exact source→DB delivery remains separate. Fresh connected validation is still required; the source↔DB timing gate remains required for true sensor→DB latency.
P2 Fabric performance/overhead	Formal procedure written	Use execution/02a-fabric-performance-overhead.md; remaining work is reproducible load generation, commit/block-size capture, and rehearsal against the commissioned Fabric path.

Area	Current project state	Required action
R1 Internet interruption	Formal procedure written	Use execution/08a-internet-interruption-recovery.md; rehearse and freeze the exact external-Internet injection method against live topology. It cannot be run while the sensor/gateway are intentionally offline.
R2 Fabric consistency/recovery	Formal procedure written	Use execution/08b-fabric-consistency-reconciliation.md; retain the same ten pending records and query-before-resubmission evidence. Rehearse the Fabric-unavailability fixture before counted testing.
A1 authentication	Substantially aligned	Keep 90 attempts and response/state-change metrics.
A2 endorsement policy	Formal procedure written	Use execution/03a-fabric-endorsement-policy.md; a Fabric-team-approved controlled missing-endorsement fixture is still required before counted testing.
S1 replay/spoofing	Substantially aligned	Keep gateway-reception validity gate.
S2 application-layer duplicate/replay	Chapter-internal methodology gap	Keep explicitly BLOCKED_BY_METHODOLOGY until the methodology supplies a count, injection procedure, expected decision, metrics, and pass rule; do not substitute I3 or ordinary DB deduplication silently.
F1/F2 flooding	Substantially aligned	Keep 18 runs, fixed rates after pilot, explicit bandwidth + recovery metrics.
I1/I2 integrity	Substantially aligned	Keep 40 application/post-storage attempts.
I3 blockchain duplicate/overwrite	Formal procedure written	Use execution/05a-blockchain-duplicate-overwrite.md; rehearse the dedicated fixture namespace and ledger-query evidence path before counted testing.
T1/T2 traceability	Substantially aligned	Keep 20 trials / 60 records and complete-linkage criteria.

Until the affected execution manuals are reconciled, **this file supersedes their conflicting test counts, conditions, and metric requirements**. It does not supersede current live infrastructure details.

Recorder / observability implications

The research recorder already captures much of the required raw material: source transmissions, gateway and ChirpStack evidence, database exports, Fabric outbox state, service logs, CPU/memory, and network counters. Before formal testing, verify or add explicit derivations/fixtures for the newly authoritative gaps:

- Fabric workload generator and 1/15 s, 1, 5, 10 TPS labels;
- Fabric submission/commit timestamps and valid-commit status;
- transaction size and block size;
- with-vs-without-Fabric latency/CPU/memory comparison;
- Internet-interruption phase markers and connectivity-state proof;

- explicit PENDING/VERIFIED/FAILED reconciliation evidence and per-record hashes;
- endorsement-policy violation/recovery evidence;
- blockchain duplicate/overwrite attempts and original-hash preservation;
- an explicit S2 application-layer duplicate/replay fixture and summary schema **only after** its methodology is formally resolved;
- network bandwidth as RX/TX raw counters plus derived Mbps;
- metric numerator/denominator columns in every summary so percentages remain auditable;
- the formal P1 PDR extractor is implemented in `summarize-run.py`: unique accepted target-device `deduplication_ids` across both HA ChirpStack logs / captured scheduled `SENSOR_TX` attempts; a scheduled source-side send failure remains in the denominator. It withholds missing or impossible observation boundaries rather than normalizing them; validate it on a fresh connected counted run before final Chapter IV use. The exact source→database percentage remains database delivery/correlation only.

Grafana/testing-monitor panels are observation aids. Formal results must be derived from sealed raw evidence and reproducible calculations, not copied visually from a dashboard.

Canonical source: [deployment/technology-guides/19-research-recorder-and-test-automation.md](#). For any mutable production-status statement, check the dated baseline and verified live state.